

Digitale Codierungen | DigCod

Zusammenfassung

INHALTSVERZEICHNIS

1. Vorwort	4
2. Stellenwertsystem	4
2.1. Konversion	4
2.2. Nachkommastellen	4
2.3. Subtraktion durch Addition	5
3. Dualsystem	5
3.1. Arithmetik	5
3.1.1. Komplementbildung (Zweierkomplement)	5
3.1.2. Subtraktion	5
3.1.3. Addition	5
3.1.4. Multiplikation	5
3.1.5. Division	6
3.2. Wertebereich	6
3.3. Bit	6
3.4. Präfixe	6
3.4.1. Multiplikation/Division	6
4. Codierungen	7
4.1. Warum reicht eine einzige Zahlendarstellung nicht aus?	7
4.2. Vorzeichen	7
4.2.1. Betrag mit Vorzeichen	8
4.2.2. (b-1) Komplement / Einerkomplement	8
4.2.3. (b) Komplement / Zweierkomplement	8
4.2.4. Exzess-Codierung (Bias-Code)	9
4.3. Gleitkommazahlen	10
4.3.1. Fixkommazahl	10
4.3.2. Allgemeiner Wertebereich	11
4.3.3. Auflösung	11
4.3.4. Arithmetik	11
4.4. Gleitkomma	12
4.4.1. Sonderfälle	13
4.4.2. Präzision	13
4.4.3. Addition	13
4.4.4. IEEE 754 - Single vs. Double	14
4.5. Text	14
4.5.1. ASCII	15
4.5.2. Unicode	15
5. Boolsche Logik	18
5.1. Terme	18
5.2. Normalformen	18
5.2.1. Disjunktive Normalform	18
5.2.2. KV-Diagramm	18

5.3. NAND (Not AND)	19
5.4. Von Logik zur Arithmetik	19
5.5. Definition $\mathbb{Z}_2$	20
5.5.1. Bitfolge Darstellungen	20
5.6. Fun games	21
6. Wahrscheinlichkeit	21
6.1. Ergebnismenge	22
6.2. Eigenschaften	22
6.3. Wahrscheinlichkeitsdefinition nach Laplace	22
6.4. Kombinatorik	23
6.4.1. Permutation mit Wiederholung	23
6.4.2. Permutation ohne Wiederholung	24
6.4.3. Variation mit Wiederholung	24
6.4.4. Variation ohne Wiederholung	24
6.4.5. Kombination mit Wiederholung	24
6.4.6. Kombination ohne Wiederholung	25
6.4.7. Übersicht Auswahl	25
6.4.8. Hypergeometrische Verteilung	25
6.5. Bitfehler	25
6.6. Gesamtwahrscheinlichkeit	26
6.7. Binomialverteilung	26
6.8. Bedingte Wahrscheinlichkeit	27
6.8.1. Zusammenhang mit Multiplikationsregel	28
6.8.2. Bayes-Theorem	28
6.9. Unabhängigkeit von Ereignissen	28
7. Informationstheorie	29
7.1. Informationsgehalt	29
7.2. Entscheidungsgehalt	29
7.3. Entropie	30
7.4. Redundanz	30
7.5. Codierung der Zeichen	31
7.5.1. Codewortlänge	31
7.5.2. Mittlere Codewortlänge	31
7.5.3. Redundanz des Codes	31
7.5.4. Präfixcodes	32
7.6. Shannon'sches Codierungstheorem	32
7.7. Diskrete Quellen	32
7.7.1. Quellen ohne Gedächtnis	32
7.7.2. Quellen mit Gedächtnis	32
8. Quellencodierung	33
8.1. Datenkomprimierung	33
8.1.1. Huffman-Codierung	34
8.1.2. Run-Length-Encoding (RLE)	35
8.1.3. Lempel-Ziv-Codierung (LZ77)	36
8.1.4. Lempel-Ziv-Welch-Komprimierung (LZW)	37
8.1.5. Kompressionsrate	38
8.2. Verschlüsselung	38
8.2.1. Substitutionsverfahren	38

8.2.2. Transpositionsverfahren	39
8.2.3. Polyalphabetisches Verfahren	39
8.2.4. Probleme	40
8.2.5. Moderne symmetrische Verschlüsselung	40
8.2.6. Asymmetrische Verschlüsselung	41
9. Kanalmodell	43
9.1. Kanalmatrix	43
9.1.1. Generalisierung	44
9.1.2. Nicht gestörter Kanal	44
9.1.3. Vollständig gestörter Kanal	44
9.1.4. Verlustfreie (rauschbehaftete) Matrix	44
9.2. Maximum-Likelihood-Verfahren (ML)	44
9.3. Maximum-A-Posteriori-Verfahren (MAP)	45
9.4. Entropie	45
9.4.1. Eingangswahrscheinlichkeit	45
9.4.2. Ausgangswahrscheinlichkeit	45
9.4.3. Gemeinsame Entropie / Verbundentropie	46
9.4.4. Entropie am Kanaleingang	46
9.4.5. Entropie am Kanalausgang	46
9.4.6. Äquivokation vs. Irrelevanz	46
9.4.7. Äquivokation	47
9.4.8. Irrelevanz	47
9.4.9. Transinformation	47
9.4.10. Maximale Symbolrate	48
9.4.11. Zusammenhänge	49
9.5. Fehlererkennung und Fehlerkorrektur	49
9.5.1. Blockcodes	49
9.5.2. Coderate	49
9.5.3. Coderaum	50
9.5.4. Hamming-Gewicht und Hamming-Distanz	50
9.5.5. Fehlererkennung	50
9.5.6. Fehlerkorrektur	51
9.5.7. 1D-Parity	51
9.5.8. 2D-Parity	51
9.5.9. Korrigierkugeln	51
9.5.10. Mögliche / gültige Codewörter	52
9.5.11. Hamming-Schranke	52
9.5.12. Kontrollmatrix und Codebedingung	53
9.5.13. Konstruktion gültiger Codewörter	53
9.5.14. Fehlersyndrom	54
10. Qubit	54
11. CPU	54
Bibliografie	55

1. VORWORT

Credit where credit is due [\[1\]](#)

2. STELLENWERTSYSTEM

<i>Begriff</i>	<i>Bedeutung</i>
Basis (Radix)	$E \in \mathbb{N}, R \geq 2$
Ziffernmenge	$T_R = \{0, 1, \dots, R-1\}$
Darstellung einer Zahl (Polynom)	$N_R = \sum_{i=0}^n d_i R^i$
Stellenwertigkeit an der Position i	$d_i \in Z_R, R^i$

Beispiele

Dezimalsystem – $R = 10$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ – $110_{10} = 0d110$	Oktalsystem – $R = 8$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7\}$ – $110_8 = 0o110 = 72_{10}$
Dualsystem – $R = 2$ – $Z_{10} = \{0, 1\}$ – $0110_2 = 0b0110 = 6_{10}$	Hexadezimalsystem – $R = 16$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$ – $2D_{16} = 0x2D = 45_{10}$

2.1. KONVERSION

Dezimal- zu Dualsystem:

- Rechts shift ist eine Division durch 2
- Es entsteht nur dann ein Rest, wenn die Bitstelle $2^0 = 1$ ist.

Beispiel: 25_{10}

Resultat: $0001\ 1001_2$

<i>Zähler</i>	<i>Nenner</i>	<i>Resultat</i>	<i>Rest</i>	
25	2	12	1	} ↑
12	2	6	0	
6	2	3	0	
3	2	1	1	
1	2	0	1	

Dual- zu Dezimalsystem bei Nachkommastellen:

- Links shift ist eine Multiplikation mit 2

Beispiel: 0.1875_{10}

Resultat: $0.001\ 1_2$

<i>a</i>	<i>b</i>	<i>Resultat</i>	<i>Rest</i>	
0.1875	2	0.375	0	} ↓
0.375	2	0.75	0	
0.75	2	1.5	1	
0.5	2	1	1	

2.2. NACHKOMMASTELLEN

Erweiterung der Darstellung einer Zahl: $N_R = d_n R^n + \dots + d_{10} R^0 + d_{-1} R^{-1} + \dots + d_{-m} R^{-m}$

Beispiel: $(101.01)_2 = 1 * 2^2 + 0 * 2^1 + 1 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2} = 4 + 1 + \frac{1}{4} = 5.25_{10}$

2.3. SUBTRAKTION DURCH ADDITION

Beispiel: $753 + 247 = 1000$

Bei 1000 einen Überlauf (mod 1000): $753 + 247 \equiv 0 \Leftrightarrow 753 \equiv -247 \pmod{1000}$

Dann wäre $620 - 247 \equiv 620 + 753 = 1373 \equiv 373 \pmod{1000}$

3. DUALSYSTEM

3.1. ARITHMETIK

3.1.1. Komplementbildung (Zweierkomplement)

- 1) Rechnen in n Bit $\Rightarrow \pmod{2^n}$
 - Übertrag aus dem MSB wird verworfen (gerechnet wird in $\mathbb{Z}_{2^n}$)
- 2) $-b$ als Zweierkomplement
 - Additives Inverses von $b \pmod{2^n}$ ist: $-b \equiv 2^n - b \pmod{2^n}$
 - Praktische Berechnung:
 - Bits invertieren: $\sim b$
 - $+1$ addieren
 - $2K(b) = \sim b + 1$

3.1.2. Subtraktion

Subtraktion wird durch Addition des Komplements ersetzt

$$a - b \equiv a + 2K(b) \pmod{2^n}$$

Beispiel 1 : $13_{10} - 5_{10}$

8 Bit, wobei $13_{10} = 0000\ 1101_2$, $5_{10} = 0000\ 0101_2$

Zweierkomplement von 5_{10} :

– invertieren: $1111\ 1010_2$

– $+1$: $1111\ 1011_2$

Addieren: $0000\ 1101_2 + 1111\ 1011_2 = 0001\ 0000\ 1000_2$

MSB-Übertrag weg: $0000\ 1000_2 = 8_{10}$

Also: $13_{10} - 5_{10} = 8_{10}$

3.1.3. Addition

TODO:

signed, unsigned

3.1.4. Multiplikation

TODO:

signed, unsigned

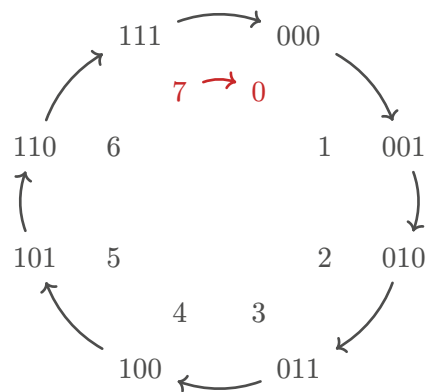
3.1.5. Division

TODO:

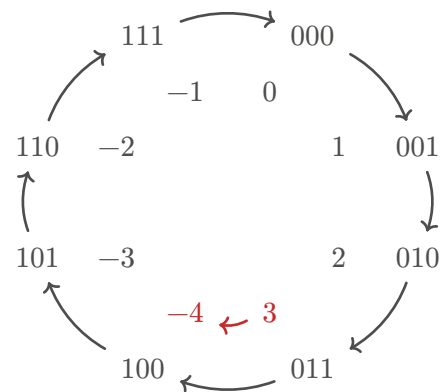
signed, unsigned

3.2. WERTEBEREICH

Graphische Veranschaulichung für Wortbreite von 3 Bit



unsigned: Überlauf von 7 auf 0



signed: Überlauf von 3 auf -4

3.3. BIT

Begriff	Bedeutung
set bit (gesetztes Bit)	1
cleared bit (gelöschtes Bit)	0
LSB	Least Significant Bit (Bit 0)
MSB	Most Significant Bit (Bit $n - 1$) in einer n -stelligen Binärzahl
Nibble	Binärzahl mit 4 Bit
Oktett	Binärzahl mit 8 Bit
Byte	Oktett

3.4. PRÄFIXE

Binär	Näherung	Binärpräfix	Dezimal	Dezimalpräfix
2^{10}	$1.024 \cdot 10^3$	Ki - Kibi	10^3	K - Kilo
2^{20}	$1.049 \cdot 10^6$	Mi - Mebi	10^6	M - Mega
2^{30}	$1.074 \cdot 10^9$	Gi - Gibi	10^9	G - Giga
2^{40}	$1.100 \cdot 10^{12}$	Ti - Tebi	10^{12}	T - Tera
2^{50}	$1.126 \cdot 10^{15}$	Pi - Pebi	10^{15}	P - Peta
2^{60}	$1.153 \cdot 10^{18}$	Ei - Exbi	10^{18}	E - Exa

3.4.1. Multiplikation/Division

- 1) Umformen in Potenzschreibweise
- 2) Addition der Exponenten
- 3) Umformen in Präfixschreibweise

Beispiel: $128K \cdot 64M = 2^7 \cdot 2^{10} \cdot 2^6 \cdot 2^{20} = 2^{17+26} = 2^{43} = 2^3 \cdot 2^{40} = 8T$

4. CODIERUNGEN

Erst wenn man die Codierung kennt, kann man daten richtig interpretieren.

4.1. WARUM REICHT EINE EINZIGE ZAHLENDARSTELLUNG NICHT AUS?

- Negative Zahlen
 - Vorzeichen-Bit
 - Einerkomplement
 - Exzess-Code
- Sehr grosse und kleine Zahlen
 - Fixkomma ist unflexibel
 - 334 Milliarden + 0.00001
- Genauigkeit
 - 0.1 ist im Dualsystem nicht exakt darstellbar
 - Rundungsfehler unvermeidbar
- Nicht nur Zahlen
 - Textdarstellung
 - Zeichenkodierungen

4.2. VORZEICHEN

Jede Codierung optimiert eine andere Eigenschaft

- Betrag mit Vorzeichen: Intuition
- Einerkomplement: Einfache Negation
- Zweierkomplement: Arithmetische Effizienz
- Exzess: Vergleichbarkeit/Sortierung

<i>Binär</i>	<i>Betrag mit V.</i>	<i>EinerK.</i>	<i>ZweierK.</i>	<i>C_{ex,-8,4}</i>
0000	0	0	0	-8
0001	1	1	1	-7
0010	2	2	2	-6
0011	3	3	3	-5
0100	4	4	4	-4
0101	5	5	5	-3
0110	6	6	6	-2
0111	7	7	7	-1
1000	0	-7	-8	0
1001	-1	-6	-7	1
1010	-2	-5	-6	2
1011	-3	-4	-5	3
1100	-4	-3	-4	4
1101	-5	-2	-3	5

1110	-6	-1	-2	6
1111	-7	0	-1	7

4.2.1. Betrag mit Vorzeichen

Erstes Bit signalisiert, ob Zahl positiv (0) oder Negativ (1) ist.

Problem: Bekannte Rechenregeln funktionieren nicht.

$$\begin{aligned}
 -19_{10} + 1_{10} &= -18_{10} \\
 1001\ 0011_2 + 0000\ 0001_2 &= 1001\ 0100_2 = -20_{10}
 \end{aligned}$$

Beispiel 2 : Codierung Betrag mit Vorzeichen

Gegeben	Dezimal $\rightarrow$ Binär	Ergebnis
$x = -3_{10}$	$3_{10} = x011_2$	$-3_{10} = 1011_2$

Beispiel 3 : Decodierung Betrag mit Vorzeichen

Gegeben	Binär $\rightarrow$ Dezimal	Ergebnis
$x = 1001_2$	$x001_2 = 1_{10}$	$1001_2 = -1_{10}$

4.2.2. (b-1) Komplement / Einerkomplement

Von allen bits wird das Komplement gebildet.

Problem: $0000\ 0000_2 = 1111\ 1111_2 = 0_{10}$

Beispiel 4 : Codierung (b-1) Komplement

Gegeben	Dezimal $\rightarrow$ Binär	Komplement	Ergebnis
$x = -5_{10}$	$5_{10} = 0101_2$	$0101_2 \Rightarrow 1010_2$	$-5_{10} = 1010_2$

Beispiel 5 : Decodierung (b-1) Komplement

Gegeben	Komplement	Binär $\rightarrow$ Dezimal	Ergebnis
$x = 1011_2$	$1011_2 \Rightarrow 0100_2$	$0100_2 = 4_{10}$	$1011_2 = -4_{10}$

4.2.3. (b) Komplement / Zweierkomplement

Nach der Komplementbildung wird 1 addiert.

Beispiel 6 : Codierung (b) Komplement

Gegeben	Dezimal $\rightarrow$ Binär	Komplement	+1	Ergebnis
$x = -5_{10}$	$5_{10} = 0101_2$	$0101_2 \Rightarrow 1010_2$	$0101_2 \Rightarrow 1011_2$	$-5_{10} = 1011_2$

Beispiel 7 : Decodierung (b) Komplement

Gegeben	-1	Komplement	Binär → Dezimal	Ergebnis
$x = 1111_2$	$1111_2 \Rightarrow 1110_2$	$1110_2 \Rightarrow 0001_2$	$0001_2 = 1_{10}$	$1111_2 = -1_{10}$

4.2.4. Exzess-Codierung (Bias-Code)

Darstellung vorzeichenbehafteter Zahlen durch **Verschiebung des Wertebereichs**.

Statt negativer Zahlen wird ein **Offset (Bias)** addiert

Bedeutung 1 : $C_{ex,-B,n}(x)$

C_{ex} = Exzess-Codierung
 $-B$ = Bias (negativer Überhang zur 0)
 n = Länge der binären Schreibweise
 x = zu codierende Zahl

Gespeichert wird: $c = x + B$

Decodierung: $x = c - B$

Gegeben	Codierung	Decodierung
Basis $b = 2$ Wortbreite n Bias B typischerweise: $B = 2^{n-1} - 1$	$C_{ex,-B,n}(x) = x + B$ mit $x \in [-B, 2^n - 1 - B]$	$x = c - B$

Beispiel 8 : Codierung Bias-Code**Gegeben**

Wortbreite: $n = 8$ Bit
 Bias: $B = 2^7 - 1 = 127$
 Zu codierende Zahl: $x = -5$

Bias addieren

$$c = x + B = -5 + 127 = 122$$

Dezimal → Binär

$$122_{10} = 0111\ 1010_2$$

Ergebnis

$$C_{ex,-127,8}(-5) = 0111\ 1010_2$$

Beispiel 9 : Decodierung Bias-Code**Gegeben**

Wortbreite: $n = 8$ Bit
 Bias: $B = 2^7 - 1 = 127$
 Bitmuster: $C_{ex,-127,8} = 0111\ 1010_2$

Binär → Dezimal

$$0111\ 1010_2 = 122_{10}$$

Bias subtrahieren

$$x = c - B = 122 - 127 = -5$$

Ergebnis

$$0111\ 1010_2 \Rightarrow -5_{10}$$

4.3. GLEITKOMMAZAHLEN**4.3.1. Fixkommazahl**

Skalierte Ganzzahl

Pro	Contra
Einfache Implementierung	Fester Wertebereich
Deterministische Genauigkeit	Feste Auflösung
Keine Exponentendarstellung	Überlauf bei grossen Zahlen
Schnelle Berechnung	Ungeeignet für stark unterschiedliche Grössenordnungen

Bedeutung 2 : $C_{FK,k,n}(x) = x \cdot 2^k$

C_{FK} = Fixkomma-Codierung

k = Anzahl Nachkommabits

n = Länge der binären Schreibweise - daraus ergibt sich die Anzahl der Vor-Kommastellen: $n - k$

x = zu codierende Zahl

I = Ganzzahl

Beispiel 10 : Codierung Fixkommazahl**Gegeben**

Wortbreite: $n = 8$ Bit

Nachkommabits: $k = 4$

Zu codierende Zahl: $x = 3.25$

Skalieren

$$I = x \cdot 2^k = 3.25 \cdot 16 = 52$$

Dezimal $\rightarrow$ Binär

$$52_{10} = 0011\ 0100_2$$

Ergebnis

$$C_{FK,4,8}(3.25) = 0011\ 0100_2$$

Beispiel 11 : Decodierung Fixkommazahl**Gegeben**

Wortbreite: $n = 8$ Bit

Nachkommabits: $k = 4$

Bitmuster: $C_{FK,4,8}(3.25)$

Binär $\rightarrow$ Dezimal

$$0011\ 0100_2 = 52_{10} = I$$

Skalieren

$$x = \frac{I}{2^k} = \frac{52}{16} = 3.25$$

Ergebnis

$$0011\ 0100_2 \Rightarrow 3.25_{10}$$

4.3.2. Allgemeiner Wertebereich

Wertebereich bei n Bit und k Nachkommabits

	Ganzzahlbereich	Reeller Bereich
unsigned	$I \in [0, 2^n - 1]$	$x \in \left[0, \frac{2^n - 1}{2^k}\right]$
signed	$I \in [-2^{n-1}, 2^{n-1} - 1]$	$x \in \left[-\frac{2^{n-1}}{2^k}, \frac{2^{n-1} - 1}{2^k}\right]$

Beispiel 12

Gegeben

Wortbreite: $n = 8$ Bit

Nachkommabits: $k = 3$

Darstellung im Zweierkomplement (= inkl. Vorzeichen)

Ganzzahlbereich

$$I \in [-2^{8-1}, 2^{8-1} - 1] = [-128, 127]$$

Reeller Bereich

$$x = \frac{I}{2^k} = \frac{I}{8}$$

$$x_{\min} = \frac{-128}{8} = -16$$

$$x_{\max} = \frac{127}{8} = 15.875$$

Ergebnis

$$x \in [-16, 15.875]$$

4.3.3. Auflösung

Kleinst darstellbarer Schritt: $\Delta x = 2^{-k}$

Absoluter Fehler: $E_{\text{abs}} = |x_{\text{korrekt}} - x_{\text{gerundet}}|$

Relativer Fehler: $E_{\text{rel}} = \frac{|x_{\text{korrekt}} - x_{\text{gerundet}}|}{x_{\text{korrekt}}}$

4.3.4. Arithmetik

Addition, Subtraktion und Zweierkomplement sind wie bei Ganzzahlen

– Addiert man zwei Fixkommazahlen $z_0 + z_1$ mit $k_0 > k_1$, so muss z_1 um $k_0 - k_1$ Bits nach rechts geschoben werden.

Multiplikation und Division verschieben das Komma

– Multipliziert man zwei Fixkommazahlen $z_0 \cdot z_1 = z_2$, dann ist $k_2 = k_0 + k_1$

k muss für jede Zahl berücksichtigt werden

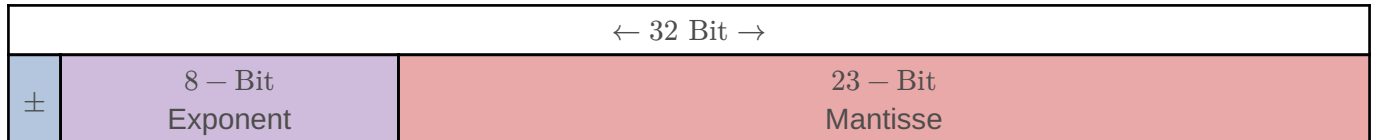
- Wird von den meisten Programmiersprachen kaum unterstützt
- Meist von Hand in Kommentaren
- Limitiert die Zahlen auf Bereiche, die zur Compilezeit bekannt sind
- unflexibel und fehleranfällig, aber performant

4.4. GLEITKOMMA

Bedeutung 3 : $\pm m \cdot 2^e$

m = Mantisse (Signifikand)

e = Exponent



$$\pm(1 + \text{Mantisse}) \cdot 2^{\text{Exponent} - 127}$$

Beispiel 13 : Codierung Gleitkommazahl

Gegeben

Zu codierende Zahl: $x = -42.625$

Vorzeichenbit bestimmen

x ist negativ, somit eine 1

Mantisse Dezimal → Binär

$$\underbrace{101010}_{42} . \underbrace{101}_{0.625}$$

Komma verschieben

$$101010.101_2 \cdot 2^0 = 1.01010101_2 \cdot 2^5$$

Runden

Bei Überfluss: falls vorherige Bit eine 1 ist, aufrunden, ansonsten kürzen (in diesem Beispiel nicht nötig).

Bias addieren

$$5 + 127 = 132$$

Exponent Dezimal → Binär

$$132_{10} \Rightarrow 1000\ 0100_2$$

Zusammensetzen

$$1 \mid 1000\ 0100 \mid (1)010\ 1010\ 1000\ 0000\ 0000\ 0000 \Rightarrow 1100\ 0010\ 0010\ 1010\ 1000\ 0000\ 0000\ 0000_2$$

Beispiel 14 : Decodierung Gleitkommazahl

Gegeben

Bitmuster: $0100\ 0001\ 0011\ 0110\ 0000\ 0000\ 0000\ 0000_2$

Komponenten extrahieren

0 | 1000 0010 | (1)011 0110 0000 0000 0000 0000

Erstes Bit

0 = Zahl ist positiv

Exponent Binär → Dezimal

$1000\ 0010_2 \Rightarrow 130_{10}$

Bias subtrahieren

$130 - 127 = 3$

Mantisse Binär → Dezimal

$011\ 0110\ 0000\ 0000\ 0000\ 0000_2 \Rightarrow 0.40625_{10}$

Zusammensetzen

$+1 \cdot (1 + 0.40625) \cdot 2^3 = 11.25$

4.4.1. Sonderfälle

Fall	Vorzeichenbit	Exponent	Mantisse	Beispielwert
Positive Null	0	0000 0000	000 0000 0000 0000 0000 0000	+0
Negative Null	1	0000 0000	000 0000 0000 0000 0000 0000	-0
Positive Unendlichkeit	0	1111 1111	000 0000 0000 0000 0000 0000	$+\infty$
Negative Unendlichkeit	1	1111 1111	000 0000 0000 0000 0000 0000	$-\infty$
NaN	0 oder 1	1111 1111	mindestens ein Bit 1	Nicht darstellbare Werte
Subnormale Zahlen	0 oder 1	0000 0000	mindestens ein Bit 1	$\approx 1.4 \cdot 10^{-45}$ (positiv)
Normalisierte Zahlen	0 oder 1	alles ausser 0000 0000 und 1111 1111	beliebig	Alle regulären Werte

4.4.2. Präzision

Viele Zahlen können nicht präzise dargestellt werden

- $0.1_{10} = 0.0001100110011...$
 - Abbruch nach 23 Bits
 - Beispiel: $0.1 + 0.2 \neq 0.3$
- Präzision für «Single Precision»: $\varepsilon \approx 2^{-23} \approx 10^{-7}$
 - ε ist die kleinste relative Differenz nahe 1

4.4.3. Addition

- 1) Hidden Bit ergänzen
- 2) Exponenten vergleichen
 - Wenn unterschiedlich: Bei kleinerer Zahl Mantisse nach rechts schieben

- 3) Vorzeichen berücksichtigen
 - Gleiche Vorzeichen: Addieren
 - Unterschiedliche Vorzeichen: Subtrahieren
- 4) Addition/Subtraktion der Signifikanden
- 5) Falls Carry = 1: Normalisieren
 - Erhöhe Exponent um 1
 - Schiebe Signifikand um 1 nach rechts
- 6) Hidden Bit entfernen

Beispiel 15 : Gleitkomma addition: $x = 1.5, y = 0.75$

- 1) Hidden Bit ergänzen

$$x' = 0 \mid 0111 \ 1111 \mid (1)100 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

$$y' = 0 \mid 0111 \ 1110 \mid (1)100 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

- 1) Exponent verschieben bei y'

$$x'' = 0 \mid 0111 \ 1111 \mid (1)100 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

$$y'' = 0 \mid 0111 \ 1111 \mid (0)110 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

- 1) Vorzeichen: beide positiv → Addition

- 2) Addition $z'' = x'' + y''$

$$z'' = 0 \mid 0111 \ 1111 \mid (10)010 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

- 1) Carry = 1: Normalisieren

$$z' = 0 \mid 1000 \ 0000 \mid (1)001 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

- 1) Hidden Bit entfernen

$$z = 0 \mid 1000 \ 0000 \mid 001 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000$$

4.4.4. IEEE 754 - Single vs. Double

Format	Bits	Vorzeichen	Exponent	Mantisse	Bias
Single	32	1	8	23	-127
Double	64	1	11	52	-1023

- Grösserer Exponent → grösserer Wertebereich
- Grössere Mantisse → höhere Präzision
 - Single Precision: $\varepsilon \approx 2^{-23} \approx 10^{-7}$
 - Double Precision: $\varepsilon \approx 2^{-52} \approx 10^{-16}$

• Interpretation

- Single ≈ 7 signifikante Dezimalstellen
- Double $\approx 15^{-16}$ signifikante Dezimalstellen

4.5. TEXT

- Text besteht aus einer endlichen Folge von Zeichen: Buchstaben, Zahlen, Satzzeichen usw.
- Alle Zeichen stammen aus einer endlichen Menge Z , dem Zeichensatz (character set)

- Jedem Zeichen kann eindeutig eine natürliche Zahl zugeordnet werden
- Ein Encoding (character encoding, Zeichenkodierung) ist eine bijektive Funktion E , die jedem Zeichen z eine natürliche Zahl zuordnet
 - $E : Z \rightarrow \{0, 1, \dots, |Z| - 1\}$
 - Text mit n Zeichen $z_0, \dots, z_{n-1}$ kann als endliche Folge von kodierten Zeichen beschrieben werden

4.5.1. ASCII

American Standard Code for Information Interchange

- Grösse des Zeichensatzes: $2^7 = 128 \rightarrow 7$ Bit (nicht 8 Bit!)
- 8-Bit-ASCII sind Extended ASCII-Varianten und nicht standardisiert, sondern Codepages – bspw.:
 - ISO-8859-1 (Latin-1)
 - für westeuropäische Sprachen mit zusätzlichen Buchstaben wie ä, ö, ü, é usw.
 - Windows-1252
 - Microsoft-Codepage – weitgehend wie ISO-8859-1 mit typografischen Zeichen wie €, “ usw.
 - Codepage 437
 - IBM-PC-Zeichensatz mit grafischen Symbolen, Linienzeichen und Sonderzeichen
- Enthält druckbare (darstellbare) Zeichen und (nicht darstellbare) Steuerzeichen (0x00=NUL, 0x07=BEL, ...)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL 00	SOH 01	STX 02	ETX 03	EOT 04	ENQ 05	ACK 06	BEL 07	BS 08	TAB 09	LF 0A	VT 0B	FF 0C	CR 0D	SO 0E	SI 0F
1	DLE 10	DC1 11	DC2 12	DC3 13	DC4 14	NAK 15	SYN 16	ETB 17	CAN 18	EM 19	SB 1A	ESC 1B	FS 1C	GS 1D	RS 1E	US 1F
2	SP 20	! 21	“ 22	# 23	\$ 24	% 25	& 26	‘ 27	(28	) 29	* 2A	+ 2B	, 2C	- 2D	. 2E	/ 2F
3	0 30	1 31	2 32	3 33	4 34	5 35	6 36	7 37	8 38	9 39	: 3A	; 3B	< 3C	= 3D	> 3E	? 3F
4	@ 40	A 41	B 42	C 43	D 44	E 45	F 46	G 47	H 48	I 49	J 4A	K 4B	L 4C	M 4D	N 4E	O 4F
5	P 50	Q 51	R 52	S 53	T 54	U 55	V 56	W 57	X 58	Y 59	Z 5A	[5B	\ 5C	] 5D	^ 5E	_ 5F
6	` 60	a 61	b 62	c 63	d 64	e 65	f 66	g 67	h 68	i 69	j 6A	k 6B	l 6C	m 6D	n 6E	o 6F
7	p 70	q 71	r 72	s 73	t 74	u 75	v 76	w 77	x 78	y 79	z 7A	{ 7B	 7C	} 7D	~ 7E	DEL 7F

4.5.2. Unicode

Globale Zeichennummerierung

- ASCII-kompatibel
- 1 bis 4 Byte pro Zeichen
- Häufige (westliche) Zeichen kurz
- Seltene (westliche) Zeichen länger
- Das erste Byte verrät, wie viele Bytes folgen
- Unicode = Codepoint

– UTF-8 = Bytecodierung

4.5.2.1. Struktur

<i>Codepoint-Bereich</i>	<i>Bytes</i>	<i>Bitmuster</i>
U+0000 - U+007F	1	0xxxxxxx
U+0080 - U+07FF	2	110xxxxx 10xxxxxx
U+0800 - U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx
U+10000 - U+10FFFF	4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

- Erstes Byte zeigt Länge
- Folgebytes beginnen mit 10
- ASCII-Zeichen (0-127) bleiben unverändert

4.5.2.2. Codierung

Gegeben: Unicode-Codepoint U

- 1) Bestimme den Wertebereich → Anzahl Bytes.
- 2) Schreibe U in Binärdarstellung.
- 3) Fülle die Bits in die x-Positionen des passenden Musters ein.
- 4) Wandle in Hex um.

Beispiel 16

Gegeben

Zeichen: ä

Unicode

$$U = E4_{16} = 228_{10} = 1110\ 0100_2$$

Benötigt 8 Bit → 2 Bytes

Muster einfügen

110xxxxx 10xxxxxx ⇒

$$11000011\ 10100100 \Rightarrow 1100\ 0011\ 1010\ 0100_2 = C3\ A4_{16}$$

$$\text{UTF-8}(\text{ä}) = C3\ A4_{16}$$

4.5.2.3. Decodierung

Gegeben: Bytefolge

- 1) Lies das erste Byte.
- 2) Bestimme anhand der führenden Bits die Länge.
- 3) Entferne die Strukturpräfixe (110, 10, etc.).
- 4) Füge die restlichen Bits zusammen.
- 5) Interpretiere als Codepoint.

Beispiel 17

Gegeben

1110 0010 1000 0010 1010 1100₂

Muster entfernen

11100010 10000010 10101100

1110xxxx 10xxxxxx 10xxxxxx ⇒

0010 000010 101100 ⇒ 0010 0000 1010 1100₂ = 20 AC₁₆ = €

4.5.2.4. Encodings

UTF-16

- Meist 2 Bytes pro Zeichen
- Zeichen ausserhalb der BMP (Basic Multilingual Pane) benötigen in UTF-16 sogenannte Surrogate Pairs (4 Bytes)
- Nicht ASCII-kompatibel

Vorteil:

- Häufig effizient für asiatische Sprachen

Nachteil:

- Komplexere Handhabung

UTF-32

- Immer 4 Bytes pro Zeichen
- Direkter Zugriff auf Codepoints
- Sehr speicherintensiv

Encoding	Länge	Vorteil	Nachteil
UTF-8	1-4 Byte	ASCII-Kompatibel, effizient	Variable Länge
UTF-16	2-4 Byte	Teils effizient	Surrogate-Komplexität
UTF-32	4 Byte	Sehr einfach	Hoher Speicherbedarf

4.5.2.5. Basic Multilingual Plane (BMP)

Unicode ist als Zahlenraum definiert: $0 \leq U \leq 10\text{ FF FF}_{16}$

Das sind $1'114'111_{10}$ mögliche Codepoints. Dieser Raum ist in Planes (Ebenen) aufgeteilt.

Die BMP umfasst $U + 0000$ bis $U + \text{FFFF}$, also: $0 \leq U \leq 65535$. Das sind genau 16 Bit.

In der BMP liegen:

- ASCII
- Lateinische Schriftzeichen
- Griechisch
- Kyrillisch
- Hebräisch
- Arabisch
- Viele asiatische Zeichen
- Mathematische Symbole
- Satzzeichen

5. BOOLSCHES LOGIK

- Eine Aussage ist entweder wahr (1) oder falsch (0)
- Aussagen können logisch verknüpft werden

Kommutativität

- $x \wedge y = y \wedge x$
- $x \vee y = y \vee x$

Distributivität

- $x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$

($x \wedge y$ schreibt man auch als xy oder $x \cdot y$ oder $x \cap y$,
 $x \vee y$ als $x + y$ oder $x \cup y$)

Assoziativität

- $(x \wedge y) \wedge z = x \wedge (y \wedge z)$
- $(x \vee y) \vee z = x \vee (y \vee z)$

Absorption

- $x \vee (x \wedge y) = x$
- $x \wedge (x \vee y) = x$

x	y	$x \wedge y$	$x \vee y$	$x \oplus y$	$\neg x$
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Dualitätsprinzip: Ersetze in einer wahren Gleichung $\wedge \leftrightarrow \vee$ und $0 \leftrightarrow 1$, Ergebnis bleibt wahr.

TODO:

[De-Morgan-Gesetze](#)

5.1. TERME

Begriff	Bedeutung
Literal	Variable oder Negation einer Variablen: x_3 (positiver Literal), $\neg x_3$ oder $\overline{x_3}$ (negatives Literal)
Konjunktionsterm	Konjunktion von Literalen: $\overline{x_1}x_3x_4 = \overline{x_1} \wedge x_3 \wedge x_4$
Disjunktionsterm	Disjunktion von Literalen: $\overline{x_1} \vee x_3 \vee x_4$
Minterm	Konjunktionsterm, der alle Parameter der Funktion enthält: $x_0\overline{x_1}\overline{x_2}x_3x_4$
Maxterm	Disjunktionsterm, der alle Parameter der Funktion enthält: $x_0 \vee \overline{x_1} \vee \overline{x_2} \vee x_3 \vee x_4$

5.2. NORMALFORMEN

Standardisierte Schreibweise. Vorteile:

- aus Wahrheitstabelle konstruierbar
- Vereinfachung systematisch möglich
- Umsetzung als Schaltung (AND-OR-Structure) direkt ableitbar

5.2.1. Disjunktive Normalform

in Term ist in DNF, wenn er eine ODER-Verknüpfung von UND-Verknüpfungen ist, z.B.:

$$f(x, y, z) = (\neg x \wedge y) \vee (x \wedge z) = \neg x \wedge y \vee x \wedge z$$

Da $\wedge$ eine höhere Bindungsstärke als $\vee$ hat, sind die Klammern nicht zwingend notwendig. Die kanonische DNF (KDNF) ist die Disjunktion der Minterme.

5.2.2. KV-Diagramm

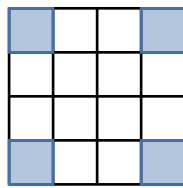
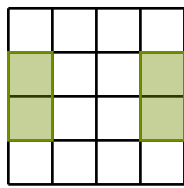
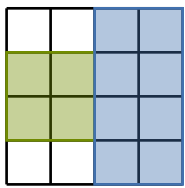
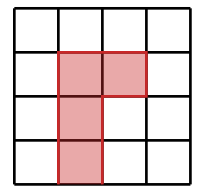
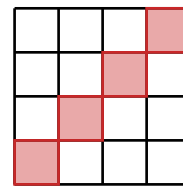
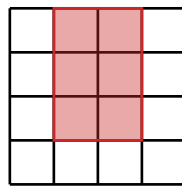
Je grösser die Blöcke, desto einfacher wird das Ergebnis. Dabei müssen aber bestimmte Regeln eingehalten werden:

- Die Blöcke müssen immer rechteckig sein und

- die Grösse einer Zweierpotenz haben, also 2, 4, 8, 16, 32, ...
- Die Blöcke können auch über den Rand hinaus gehen und mit der gegenüberliegenden Seite verbunden werden,
- Blöcke können sich teilweise überlappen. Das kann sinnvoll sein, wenn dadurch grössere Blöcke entstehen.
- Werte, die sowohl einfach als auch negiert vorkommen, werden gestrichen
- Ein Block wird nur berücksichtigt, wenn seine Einsen nicht vollständig in anderen Blöcken enthalten sind. Andernfalls entsteht ein nichtessentieller Term, der redundant ist, da andere Terme bereits die gleichen Variablenbelegungen abdecken und nicht weiter vereinfacht werden können.

TODO:

DMI

OK**NOK**

5.3. NAND (NOT AND)

Universalbaustein

$$x \downarrow y = \overline{x \wedge y} = \overline{xy}$$

Sheffer stroke

- praktisch (in CMOS schnell sowie einfach aufzubauen)
- funktional vollständig: jede Boolesche Funktion lässt sich nur mit NAND realisieren.

x	y	$x \wedge y$	$x \downarrow y$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

OP	$impl$
NOT	$\overline{x} = \overline{x \wedge x} = x \downarrow x$
AND	$x \wedge y = \overline{x \downarrow y} = (x \downarrow y) \downarrow (x \downarrow y)$
OR	$x \vee y = \overline{\overline{x \vee y}} = \overline{\overline{x} \wedge \overline{y}} = \overline{(x \downarrow x) \wedge (y \downarrow y)} = (x \downarrow x) \downarrow (y \downarrow y)$

5.4. VON LOGIK ZUR ARITHMETIK

Boolesche Logik realisiert binäre Addition

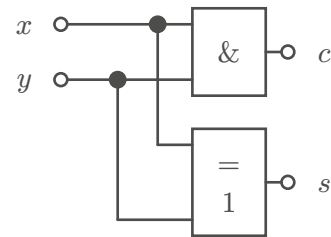
XOR bildet die Addition zweier Bits ab (im Zahlenraum mit nur 0 und 1, also mod 2), AND bildet den Übertrag ab

- $s = x \oplus y \rightarrow$ Addition mod 2
- $c = x \wedge y \rightarrow$ Übertrag

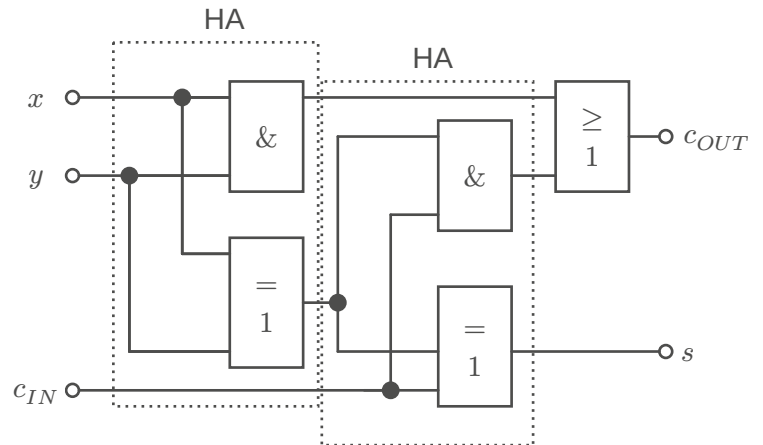
x	y	$x + y$	$x \wedge y$	$x \oplus y$
0	0	00	0	0
0	1	01	0	1
1	0	01	0	1
1	1	10	1	0

Halbaddierer (half adder)

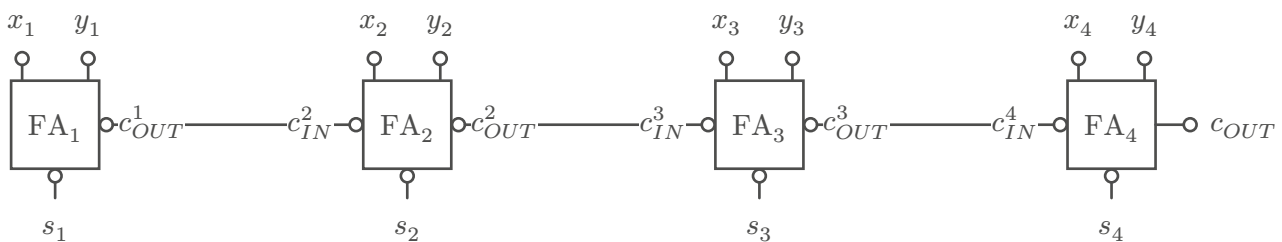
- Kann 2 einstellige Binärzahlen addieren
- Hat 2 Eingänge (x, y) und 2 Ausgänge (s, c)

**Volladdierer (full adder)**

- Hat 3 Eingänge (x, y, c_{IN})

**4-Bit Ripple carry adder**

- Jedes Bit muss auf das Carry-Bit des letzten Volladdierers warten

**Carry Look-Ahead adder**

- Reduziert propagations-delay durch komplexere hardware

5.5. DEFINITION $\mathbb{Z}_2$

$$\mathbb{Z}_2 = (\{0, 1\}, +, \cdot)$$

- Abgeschlossen: Jede Operation erzeugt wieder ein Element in $\mathbb{Z}_2$
- Es existiert das neutrale Element (0 für die Addition, 1 für die Multiplikation)
- Jedes Element ist sein eigenes additives Inverses.
- Addition und Multiplikation sind kommutativ und assoziativ.
- Es gilt das Distributivgesetz.

5.5.1. Bitfolge Darstellungen

Darstellung	Beispiel
Zahl	$0101_2 = 5_{10}$
Vektor	$(1, 0, 1)^T$
Polynom	$u^2 + 1$

5.5.1.1. Vektor

Beispiel 18 : Bitfolge als Vektor in $\mathbb{Z}_2^n$

$$v, w \in \mathbb{Z}_2^4, v = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix}, w = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

$$v + w = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \end{pmatrix} \equiv \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} \pmod{2}$$

5.5.1.2. Polynom

Polynome erlauben

- Verschiebungen elegant zu beschreiben
- Redundanz strukturiert zu erzeugen
- Generatorpolynome (zyklische Codes)
- CRC-Rechnung als Polynomdivision

Beispiel 19 : Bitfolge als Polynom in $\mathbb{Z}_2$

$$[1 \ 0 \ 0 \ 1 \ 0 \ 1]$$

$$= 1u^5 + 0u^4 + 0u^3 + 1u^2 + 0u^1 + 1u^0$$

$$= u^5 + u^2 + 1$$

u ist keine Zahl, u^k beschreibt «Bitposition k »

Polynomaddition in $\mathbb{Z}_2$

$$(u^3 + u + 1) + (u^3 + u^2)$$

$$= 2u^3 + u^2 + u + 1$$

$$= u^2 + u + 1$$

Polynommultiplikation in $\mathbb{Z}_2$

$$(u^2 + u + 1)(u + 1)$$

$$= u^3 + 2u^2 + 2u + 1$$

$$= u^3 + 1$$

5.6. FUN GAMES

[NAND Game](#)

[nand2tetris](#)

6. WAHRSCHEINLICHKEIT

In realen Systemen trifft Unsicherheit auf. Diese lässt sich nicht exakt vorhersagen, sondern nur statistisch beschreiben. Dafür verwenden wir **Wahrscheinlichkeit**.

<i>Begriff</i>	<i>Bedeutung</i>
Disjunkte Ereignisse	Ereignisse, die sich gegenseitig ausschliessen. Beispiel: z ist gerade oder ungerade
Zufallsvorgang	Ein Vorgang mit mehreren möglichen Ergebnissen, dessen Ausgang nicht sicher vorhergesagt werden kann
Zufallsexperiment	Zufallsvorgang, der geplant ist und kontrolliert abläuft

6.1. ERGEBNISMENGE

<i>Begriff</i>	<i>Bedeutung</i>
Ergebnismenge	Die Ergebnismenge eines Zufallsvorgangs umfasst alle möglichen Ausgänge des Experiments. Sie wird mit dem Symbol Ω (Omega) bezeichnet. Die Anzahl aller möglichen Ergebnisse der Ergebnismenge wird mit $ \Omega $ bezeichnet.
Ergebnis	Ein einzelner möglicher Ausgang $\omega \in \Omega$ heisst Ergebnis
Ereignis	Ein Ereignis ist eine Teilmenge der Ergebnismenge $A \subset \Omega$

Beispiel 20

Würfel: $\Omega = \{1, 2, 3, 4, 5, 6\}$

Werfen einer Münze so lange, bis Kopf erscheint: $\Omega = \{K, ZK, ZZK, ZZZK, \dots\}$

6.2. EIGENSCHAFTEN

Für jedes Ereignis A gilt $0 \leq P(A) \leq 1$.

<i>Begriff</i>	<i>Eigenschaft</i>
Sicheres Ereignis	Die Wahrscheinlichkeit der gesamten Ergebnismenge ist $P(\Omega) = 1$
Unmögliches Ereignis	Die Wahrscheinlichkeit der leeren Menge ist $P(\emptyset) = 0$
Additionsregel	Für zwei Ereignisse A und B : $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ Für disjunkte Ereignisse: $P(A \cup B) = P(A) + P(B)$
Gegenereignis	Wird notiert als $\bar{A}$ und hat die Eigenschaft $P(\bar{A}) = 1 - P(A)$.

Wahrscheinlichkeit, dass A_1 oder A_2 auftritt: $P(A_1 \vee A_2) = P(A_1) + P(A_2)$

Wahrscheinlichkeit, dass zuerst A_1 und dann A_2 auftritt: $P(A_1 \wedge A_2) = P(A_1) \cdot P(A_2)$

6.3. WAHRSCHEINLICHKEITSDEFINITION NACH LAPLACE

Wenn ein Experiment eine Anzahl verschiedener und gleich möglicher Ausgänge hervorbringen kann und einige davon als günstig anzusehen sind, dann ist die Wahrscheinlichkeit eines günstigen Ausganges gleich dem Verhältnis der Anzahl der günstigen zur Anzahl der möglichen Ausgänge.

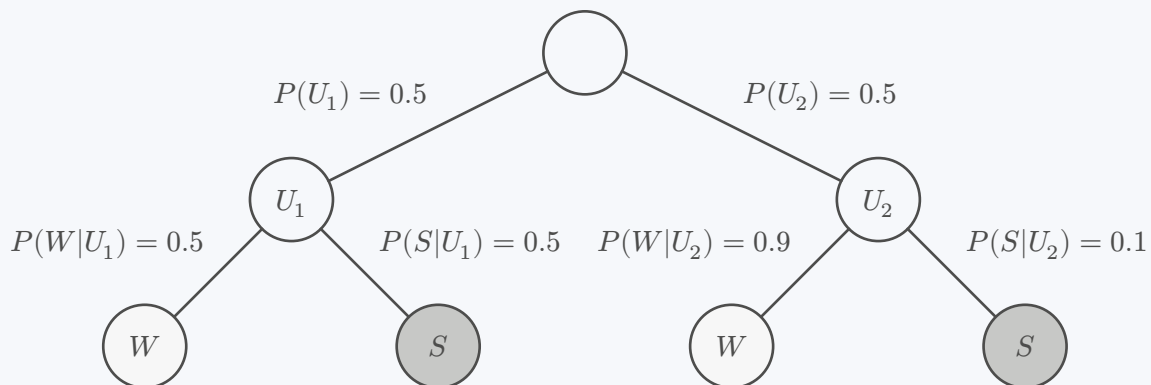
$$P(E) = \frac{\text{Anzahl günstiger Ergebnisse}}{\text{Anzahl aller möglichen Ergebnisse}} = \frac{|E|}{|\Omega|}$$

Dabei gilt:

- Ω : Ergebnismenge (alle möglichen Ergebnisse)
- $E \subseteq \Omega$: betrachtetes Ereignis
- $|E|$: Anzahl günstiger Ergebnisse
- $|\Omega|$: Anzahl aller möglichen Ergebnisse

Beispiel 21 : Urnen

Es gibt zwei Urnen, U_1 und U_2 . Urne U_1 enthält 5 schwarze und 5 weiße Kugeln und Urne U_2 enthält 9 schwarze und 1 weiße Kugel. Die Wahrscheinlichkeit eine Kugel aus U_1 oder U_2 zu ziehen, ist gleich gross (50/50). Wie gross ist nun die Wahrscheinlichkeit eine weiße Kugel zu ziehen?



$$\begin{aligned}
 P(W|U_1) \wedge P(U_1) \vee P(W|U_2) \wedge P(U_2) &= P(W|U_1) \cdot P(U_1) + P(W|U_2) \cdot P(U_2) \\
 &= 0.5 \cdot 0.5 + 0.9 \cdot 0.5 \\
 &= 0.25 + 0.45 \\
 &= 0.7
 \end{aligned}$$

Die Wahrscheinlichkeit beträgt also 70%, eine weiße Kugel zu ziehen unter der Annahme, dass jede Urne mit gleicher Wahrscheinlichkeit gewählt wird.

6.4. KOMBINATORIK

Laplace-Wahrscheinlichkeit erfordert Berechnung von Anzahlen. **Kombinatorik** ist die Mathematische Technik hierfür.

	Wiederholung	Keine Wiederholung
Anordnung	Permutation mit Wiederholung	Permutation ohne Wiederholung
Geordnete Auswahl	Variation mit Wiederholung	Variation ohne Wiederholung
Ungeordnete Auswahl	Kombination mit Wiederholung	Kombination ohne Wiederholung

6.4.1. Permutation mit Wiederholung

Anordnung von n Objekten, die **nicht** alle voneinander unterscheidbar sind (s Subsets k mit gleichen Objekten).

$$\frac{n!}{k_1! k_2! \dots k_s!}$$

Beispiel 22 : Anordnung drei roter und zwei blauer Kugeln

$$s = 2, n = 5, k_1 = 3, k_2 = 2, \text{Anzahl} = \frac{(5 \cdot 4 \cdot 3 \cdot 2 \cdot 1)}{(3 \cdot 2 \cdot 1)(2 \cdot 1)} = 10$$

6.4.2. Permutation ohne Wiederholung

Anordnung von n Objekten, die alle unterscheidbar sind.

$$n!$$

Beispiel 23 : Anordnung fünf verschiedenfarbiger Kugeln

$$\text{Anzahl} = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 5!$$

6.4.3. Variation mit Wiederholung

- Reihenfolge spielt eine Rolle
- Elemente dürfen mehrfach vorkommen

$$n^k$$

Beispiel 24 : PIN-Code mit 4 Ziffern

0000
1234
9876

Jede Position hat 10 Möglichkeiten
Anzahl = $10 \cdot 10 \cdot 10 \cdot 10 = 10^4$

6.4.4. Variation ohne Wiederholung

- Reihenfolge spielt eine Rolle
- Elemente dürfen **nicht** mehrfach vorkommen

$$\frac{n!}{(n-k)!} = \prod_n^{n-k+1} n$$

Beispiel 25 : 1., 2. und 3. Platz aus 10 Teilnehmern

$$\text{Anzahl} = 10 \cdot 9 \cdot 8$$

6.4.5. Kombination mit Wiederholung

- Reihenfolge spielt **keine** Rolle
- Elemente dürfen mehrfach vorkommen

$$\binom{n+k-1}{k} = \frac{(n+k-1)!}{(n-1)! \cdot k!}$$

Beispiel 26 : Es sollen drei von fünf verschiedenfarbigen Kugeln gezogen und wieder zurückgelegt werden.

$$\text{Anzahl} = \binom{5+3-1}{3} = \binom{7}{3} = 35$$

6.4.6. Kombination ohne Wiederholung

- Reihenfolge spielt **keine** Rolle
- Elemente dürfen **nicht** mehrfach vorkommen

$$\binom{n}{k} = \frac{n!}{(n-k)!k!} = \frac{n!}{k!(n-k)!} = \frac{\prod_n^{n-k+1} n}{k!}$$

Beispiel 27 : Lotto 6 aus 42

$$\text{Anzahl} = \binom{42}{6}$$

6.4.7. Übersicht Auswahl

Anzahl A der Möglichkeiten			
n Optionen k Auswählen		Beachtung der Reihenfolge	
		✓	×
Wiederholung erlaubt	✓	$A = n^k$	$A = \binom{n+k-1}{k}$
	×	$A = n(n-1)\dots(n-k+1) = \frac{n!}{(n-k)!}$	$A = \frac{n!}{k!(n-k)!} = \binom{n}{k}$

6.4.8. Hypergeometrische Verteilung

Die hypergeometrische Verteilung ist ein Modell, das verwendet wird, um die Wahrscheinlichkeit $P(k)$ von k Erfolgen (gewünschten Ergebnissen) aus den K gesamt möglichen Erfolgen aus N Objekten in n Ziehungen zu berechnen.

$$P(k) = \frac{\binom{K}{k} \cdot \binom{N-K}{n-k}}{\binom{N}{n}}$$

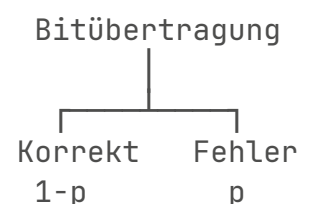
Beispiel 28

TODO:

6.5. BITFEHLER

Ein Bitfehler bezeichnet die inkorrekte Wiedergabe eines Bits während der Datenübertragung oder -speicherung. Das bedeutet, dass ein Bit von 0 zu 1 oder von 1 zu 0 fälschlicherweise geändert wird.

Die Wahrscheinlichkeit, dass ein Datenblock der Grösse n Bits bei einer Fehlerrate von p fehlerhaft ist, kann mit folgender Formel berechnet werden:



$$P(\text{fehlerhaft}) = 1 - (1 - p)^n$$

Wahrscheinlichkeit für genau k Fehler in einem Datenblock mit n Bits bei Bitfehlerrate p (Binomialverteilung):

$$P(k) = \binom{n}{k} \cdot p^k \cdot (1 - p)^{n-k}$$

Beispiel 29 : Bitfehler

Gegeben sei eine Bitfehlerrate von 10^{-5} . Wie gross ist die Wahrscheinlichkeit, dass ein Datenblock mit einer Grösse von 150 kbit fehlerhaft ist?

$$P(\text{fehlerhaft}) = 1 - (1 - 10^{-5})^{150 \cdot 10^3} \approx 0.77687$$

Was ist die Wahrscheinlichkeit für genau 2 Fehler?

$$P(2) = \binom{150 \cdot 10^3}{2} \cdot (10^{-5})^2 \cdot (1 - 10^{-5})^{150 \cdot 10^3 - 2} \approx 0.25102$$

6.6. GESAMTWAHRSCHEINLICHKEIT

Jede mögliche Fehleranordnung hat die Wahrscheinlichkeit

$$p^k (1 - p)^{n-k}$$

Es gibt $\binom{n}{k}$ solche Anforderungen. Darum gilt

$$P(X = k) = \overbrace{\binom{n}{k}}^{\text{Anzahl Kombinationen}} \underbrace{p^k}_{k \times \text{Erfolg}} \overbrace{(1 - p)^{n-k}}^{n-k \times \text{Misserfolg}}$$

Beispiel 30 : Schraubenproduktion

Bei einer Schraubenproduktion ist jede 10e Schraube fehlerhaft. Was ist die Wahrscheinlichkeit, dass 2 von 3 Schrauben OK sind?

$$P(X = 2) = \binom{3}{2} \cdot \left(\frac{9}{10}\right)^2 \cdot \left(1 - \frac{9}{10}\right)^{3-2} = \binom{3}{2} \cdot \frac{81}{100} \cdot \frac{1}{10} = 3 \cdot \frac{81}{1000} = 0.243$$

TODO:

example (slides 27)

TODO:

Restfehlerwahrscheinlichkeit

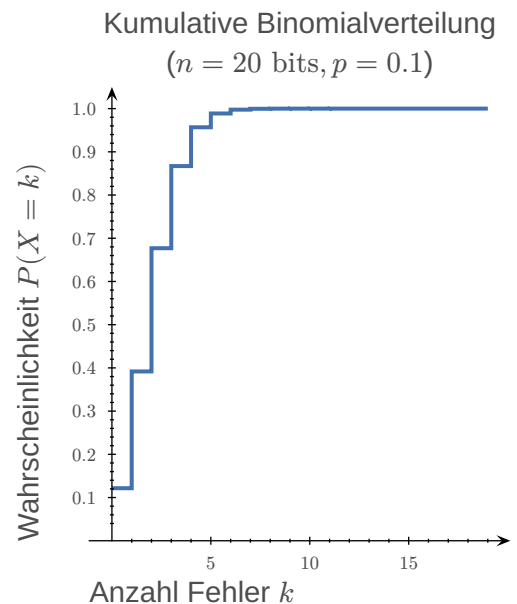
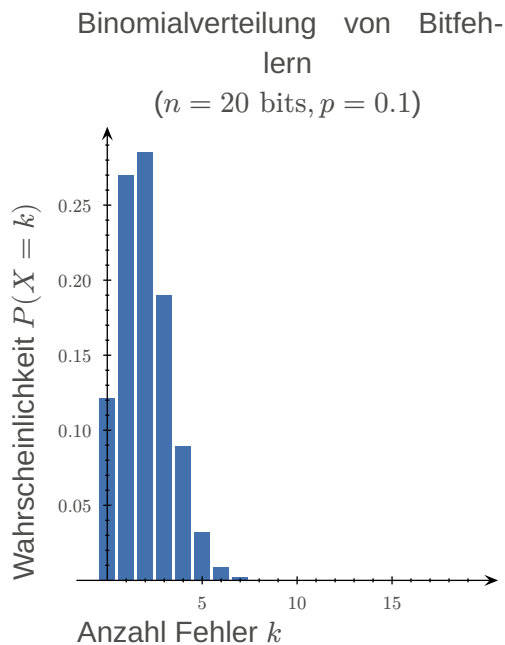
6.7. BINOMIALVERTEILUNG

$$E(X) = np$$

Die Binomialverteilung beschreibt, wie wahrscheinlich es ist, dass bei n unabhängigen Versuchen genau k Erfolge auftreten.

In unserem Kontext bedeutet das:

- n = Anzahl übertragener Bits
- p = Bitfehlerwahrscheinlichkeit
- k = Anzahl Fehler



6.8. BEDINGTE WAHRSCHEINLICHKEIT

Wie wahrscheinlich ist Ereignis A , wenn Ereignis B bereits eingetreten ist? Formelle Schreibweise: $P(A|B)$.

Allgemein gilt für die bedingte Wahrscheinlichkeit:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Voraussetzung ist dabei, dass gilt:

$$P(B) > 0$$

Die Formel bedeutet:

- Im Nenner steht die Wahrscheinlichkeit der Bedingung B .
- Im Zähler steht die Wahrscheinlichkeit, dass sowohl A als auch B eintreten.

Man betrachtet also nur noch den Teil des Wahrscheinlichkeitsraums, in dem B gilt, und fragt dann, wie gross darin der Anteil der Fälle ist, in denen zusätzlich A eintritt.

Daraus folgt die **Multiplikationsregel**:

$$P(A \cap B) = P(A|B) \cdot P(B)$$

Beispiel 31 : Würfelnwurf

$A = \text{«Die gewürfelte Zahl ist gerade»}$

$B = \text{«Die gewürfelte Zahl ist grösser als 3»}$

$$A = \{2, 4, 6\}, B = \{4, 5, 6\}, P(A) = P(B) = \frac{1}{2}$$

$$A \cap B = \{4, 6\}, P(A | B) = \frac{2}{3}$$

Die Wahrscheinlichkeit für «gerade Zahl» ist also unter der Bedingung «grösser als 3» nicht mehr $\frac{1}{2}$, sondern $\frac{2}{3}$.

6.8.1. Zusammenhang mit Multiplikationsregel

Aus der Definition folgt direkt eine wichtige Umformung:

$$P(A \cap B) = P(A|B) \cdot P(B)$$

Ebenso gilt auch:

$$P(A \cap B) = P(B|A) \cdot P(A)$$

6.8.2. Bayes-Theorem

Setzt man die beiden Ausdrücke gleich, erhält man

$$P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$$

Durch Umstellen ergibt sich das **Bayes-Theorem**:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Diese Formel erlaubt es, Wahrscheinlichkeiten umzukehren: Aus der Wahrscheinlichkeit von B unter der Bedingung A kann die Wahrscheinlichkeit von A unter der Bedingung B berechnet werden.

6.8.2.1. Satz der totalen Wahrscheinlichkeit

Angenommen ein Ereignis B kann durch mehrere verschiedene Ursachen entstehen. Seien

$$A_1, A_2, \dots, A_n$$

Ereignisse, die

- sich gegenseitig ausschließen
- zusammen den gesamten Ereignisraum bilden

Dann gilt für jedes Ereignis B :

$$P(B) = P(B|A_1) \cdot P(A_1) + P(B|A_2) \cdot P(A_2) + \dots + P(B|A_n) \cdot P(A_n)$$

Diese Formel nennt man den **Satz der totalen Wahrscheinlichkeit**. Sie beschreibt die Gesamtwahrscheinlichkeit eines Ereignisses als Summe aller möglichen Fälle, in denen es entstehen kann.

6.9. UNABHÄNGIGKEIT VON EREIGNISSEN

Zwei Ereignisse A und B heissen unabhängig, wenn das Eintreten des einen Ereignisses keinen Einfluss auf die Wahrscheinlichkeit des anderen hat. Dann gilt:

$$P(A|B) = P(A)$$

Das bedeutet: Auch wenn B bereits eingetreten ist, bleibt die Wahrscheinlichkeit von A unverändert. Setzt man das in die Multiplikationsregel ein, erhält man:

$$P(A \cap B) = P(A) \cdot P(B)$$

7. INFORMATIONSTHEORIE

Die Informationstheorie versucht, mathematisch zu messen:

- wie viel Information Ereignisse enthalten
 - ein sehr erwartbares Ereignis liefert wenig Information
 - ein überraschendes Ereignis liefert viel Information
- wie unsicher eine Informationsquelle ist
- wie effizient Informationen codiert werden können.

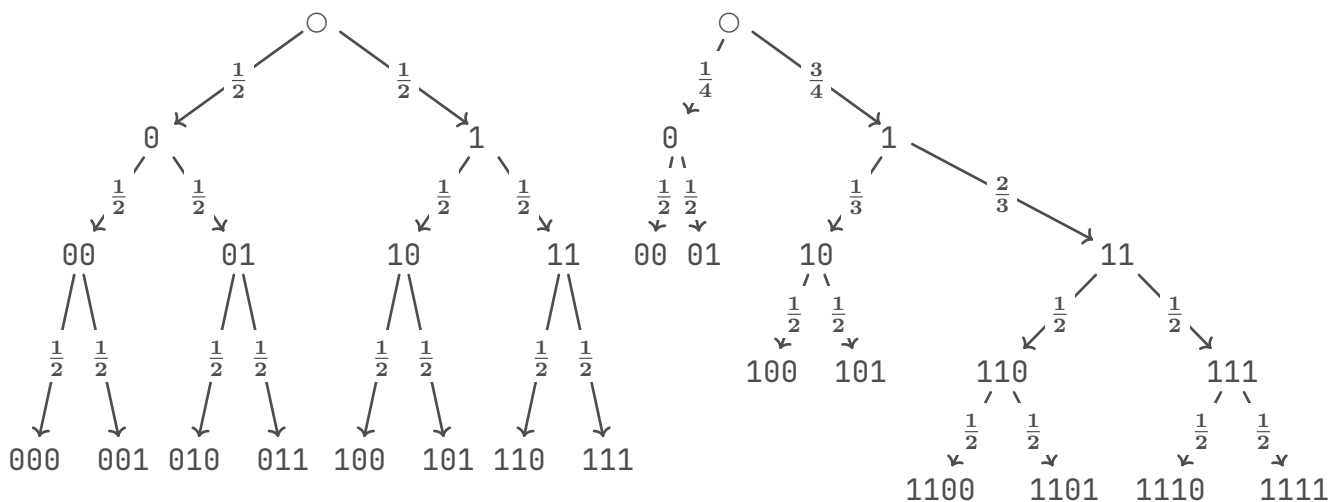
Die **Entropie** ist dabei die zentrale Grösse, die die durchschnittliche Informationsmenge einer Quelle beschreibt.

Die Einheit der Information ist das **Bit**.

7.1. INFORMATIONSGEHALT

Der Informationsgehalt $I(x_k)$ eines Symbols x_k ist ein Mass dafür, wie viel Information das Symbol trägt, basierend auf seiner Wahrscheinlichkeit des Auftretens $p(x_k)$.

$$I(x_k) = \log_2 \left(\frac{1}{p(x_k)} \right) = -\log_2(p(x_k))$$



7.2. ENTSCHEIDUNGSGEHALT

Der Entscheidungsgehalt H_0 einer Quelle gibt an, wie viel Information im Durchschnitt benötigt wird, um ein Ereignis aus N gleich wahrscheinlichen unterschiedlichen Ereignissen zu identifizieren.

$$H_0 = \log_2(N)$$

TODO:

Gleichwahrscheinliche Ereignisse

- Angenommen eine Quelle besitzt N mögliche und gleichwahrscheinliche Symbole.

Wahrscheinlichkeit eines Symbols

- Bei gleichwahrscheinlichen Symbolen gilt: $p(x) = \frac{1}{N}$

Informationsgehalt eines Ereignisses: $I(x) = -\log_2 P(x)$

- Einsetzen von $p(x) = \frac{1}{N}$: $I(x) = -\log_2\left(\frac{1}{N}\right) = \log_2 N$

Ergebnis: $I(x) = \log_2 N = H_0$

- für gleichwahrscheinliche Ereignisse.

⇒ Der Entscheidungsgehalt H_0 ist ein Spezialfall des Informationsgehalts $I(x)$ für gleichwahrscheinliche Ereignisse.

7.3. ENTROPIE

Die Entropie $H(X)$ beschreibt den durchschnittlichen Informationsgehalt / durchschnittliche Unsicherheit einer Quelle.

$$H(X) = \sum_{k=1}^N p(x_k) \cdot I(x_k) = \sum_{k=1}^N p(x_k) \cdot \log_2 \left(\frac{1}{p(x_k)} \right) = - \sum_{k=1}^N p(x_k) \cdot \log_2(p(x_k))$$

$p(x) = 1$	$p(x) = 0$	$0 \leq H(X) \leq \log_2 N$
– Ergebnis ist sicher – $I(x) = 0$	– Ergebnis tritt nie auf – Trägt 0 zur Entropie bei	– Minimum: deterministische Quelle – Maximum: gleichverteilte Quelle

Beispiel 32 : $H(X) = -(0.99 \cdot \log_2 0.99 + 0.01 \cdot \log_2 0.01) = 0.014 + 0.066 \approx 0.080$ Bit

- A liefert fast keine Information, weil es praktisch immer kommt.
- B liefert sehr viel Information, weil es extrem selten ist.
 - Aber: B kommt nur 1% der Zeit, deshalb ist der durchschnittliche Informationsgehalt klein.

Zeichen	Wahrscheinlichkeit
A	0.99
B	0.01

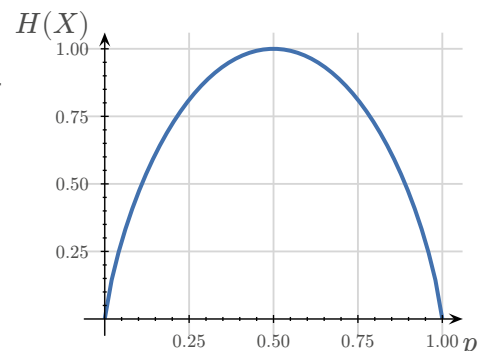
⇒ Ergebnis: ca. 0.08 Bit pro Symbol

7.4. REDUNDANZ

Die Redundanz R_q beschreibt den Anteil vorhersehbarer Information / den Unterschied zwischen dem maximal möglichen Entscheidungsgehalt und der tatsächlichen Entropie der Quelle.

$$R_q = R_{\text{abs}} = H_0 - H(X) \quad [\text{Bit/Zeichen}]$$

$$R_{\text{rel}} = \frac{R_{\text{abs}}}{H_0} = \frac{H_0 - H(X)}{H_0} = 1 - \frac{H(X)}{H_0} \quad [\%]$$



Hohe Redundanz

Quelle stark vorhersehbar

Geringe Redundanz

Quelle nahe maximaler Unsicherheit

Eine Quelle mit hoher Redundanz enthält viele regelmässige Strukturen oder Abhängigkeiten. Diese können genutzt werden, um Daten effizienter zu codieren oder zu komprimieren.

7.5. CODIERUNG DER ZEICHEN

7.5.1. Codewortlänge

Idealerweise sollte die Länge eines Codeworts dem Informationsgehalt des Symbols entsprechen:

$$L(x_k) \approx I(x_k) \approx -\log_2 p(x_k)$$

Da Codewörter jedoch aus einer **ganzen Anzahl Bits** bestehen müssen, wird aufgerundet:

$$L(x_k) = \lceil I(x_k) \rceil = \lceil -\log_2 p(x_k) \rceil$$

7.5.2. Mittlere Codewortlänge

Die mittlere Codewortlänge $L(X)$ ist definiert als der gewichtete Durchschnitt der Längen der Codewörter, wobei jedes Gewicht der Auftretenswahrscheinlichkeit des entsprechenden Symbols gleich ist.

$$L(X) = \sum_{k=1}^N p(x_k) \cdot L(x_k)$$

Es gilt für jeden Code $H(X) \leq L(X)$

Beispiel 33

Folgende Zeichen mit entsprechenden Codewörter, deren Länge und ihren Wahrscheinlichkeiten sind gegeben:

Zeichen	Codewort	Wahrscheinlichkeit p	Länge
a	0	0.3	1
b	110	0.1	3
c	1111	0.1	4
d	1110	0.2	4
e	10	0.3	2

Nun berechnen wir die mittlere Codewortlänge L mit der obigen Formel:

$$\begin{aligned}
 L &= (0.3 \cdot 1) + (0.1 \cdot 3) + (0.1 \cdot 4) + (0.2 \cdot 4) + (0.3 \cdot 2) \\
 &= 0.3 + 0.3 + 0.4 + 0.8 + 0.6 \\
 &= 2.4 \text{ bit}
 \end{aligned}$$

7.5.3. Redundanz des Codes

Die Redundanz des Codes R_c ist die Differenz zwischen der mittleren Codewortlänge und der Entropie der Quelle.

$$R_c = L - H(x)$$

Interpretation: Beschreibt, wie ineffizient die Codierung ist / Verlust durch nicht-optimalen Code.

7.5.4. Präfixcodes

Ein Code besitzt die **Präfixeigenschaft**, wenn kein Codewort der Anfang eines anderen Codeworts ist.

Präfixcodes sind wichtig, weil sie eine **eindeutige und sofortige Decodierung** ermöglichen.

Symbol	Code
A	0
B	10
C	110
D	111

7.6. SHANNON'SCHES CODIERUNGSTHEOREM

Das Shannon-Theorem beschreibt die theoretischen Grenzen der Datenkompression. Für jede Informationsquelle mit mittlerer Codewortlänge $L(X)$ und deren Entropie $H(X)$ gilt:

$$H(X) \leq L(X) < H(X) + 1$$

- Entropie ist die theoretische Mindestlänge eines Codes / Grenze der Kompression
- Praktische Codes können dieser Grenze sehr nahe kommen.

7.7. DISKRETE QUELLEN

7.7.1. Quellen ohne Gedächtnis

Diskrete Quellen ohne Gedächtnis haben Symbole, die unabhängig von vorherigen Symbolen auftreten. Jedes Symbol x_k aus einem endlichen Set tritt mit einer Wahrscheinlichkeit $p(x_k)$ auf. Verbundwahrscheinlichkeit für die beiden Zeichen x_i und x_{i+1} lautet:

$$p(x_i, x_{i+1}) = p(x_i) \cdot p(x_{i+1})$$

7.7.2. Quellen mit Gedächtnis

In der Praxis sind nur wenige Datenquellen vollständig gedächtnislos. Häufig hängt die Wahrscheinlichkeit eines Zeichens vom vorhergehenden Zeichen ab. Solche Kontextabhängigkeiten lassen sich mit bedingten Wahrscheinlichkeiten beschreiben: $p(x_{i+1} | x_i)$

Beispiel 34 : Zeichenfolgen

In deutschen und englischen Texten folgt auf den Buchstaben «q» praktisch immer «u».

$$p(u|q) \approx 1$$

7.7.2.1. Entropie

Für zwei aufeinanderfolgende Zeichen gilt:

$$H(X, Y) = \sum_{k=1}^N \sum_{i=1}^N p(x_k, y_i) \cdot \log_2 \left(\frac{1}{p(x_k, y_i)} \right)$$

Mit

$$p(x_k, y_i) = p(x_k) \cdot p(y_i | x_k)$$

ergibt sich

$$H(X, Y) = H(X) + H(Y | X)$$

Interpretation

- $H(X)$: Unsicherheit über das erste Zeichen
- $H(X, Y)$: «Verbundentropie» = Unsicherheit über zwei aufeinanderfolgende Zeichen
- $H(Y | X)$: verbleibende Unsicherheit über Y wenn X bereits bekannt ist

Da Kontextinformation Unsicherheit reduziert, gilt:

$$H(Y | X) \leq H(Y)$$

7.7.2.2. Redundanz

$$R_{\text{abs,oG}} = H_0 - H(Y)$$

$$R_{\text{abs,mG}} = H_0 - H(Y | X)$$

$$R_{\text{abs,oG}} \leq R_{\text{abs,mG}}$$

Interpretation: beschreibt, wie viel «überflüssige Struktur» in der Quelle steckt / Potenzial für Kompression.

Kontext reduziert Unsicherheit

- Entropie sinkt
- Redundanz steigt

TODO:

8. QUELLENCODIERUNG

Anwendungen:

Datenkomprimierung

- Verlustfrei
- Verlustbehaftet

Verschlüsselung

- Symmetrisch
- Asymmetrisch

8.1. DATENKOMPRIMIERUNG

Kompression ist nur möglich, wenn **Redundanz** vorhanden ist.

<i>Art der Redundanz</i>	<i>Beispiel</i>	<i>Verfahren</i>
Wiederholungen	AAAAAAAA	«Run-Length-Encoding (RLE)» (Seite 35)
Ungleiche Wahrscheinlichkeiten	häufige Zeichen	«Huffman-Codierung» (Seite 34)
Muster / Struktur	ABCABCABC	«Lempel-Ziv-Codierung (LZ77)» (Seite 36)

Verfahren zur Datenkomprimierung

Statistische Verfahren	z.B. Huffman-Codierung für die deutsche Sprache – nutzen bekannte Wahrscheinlichkeiten	Eigenart der Daten (Wahrscheinlichkeiten) werden berücksichtigt
Adaptive Verfahren	z.B. Huffman-Codierung mit gemessener Häufigkeitsverteilung – lernen Wahrscheinlichkeiten während der Codierung	
Wörterbuchbasierte Verfahren	z.B. LZ77, LZ78, LZW, DEFLATE – nutzen wiederkehrende Muster im Datenstrom	Eigenart der Daten (Wahrscheinlichkeiten) werden nicht explizit berücksichtigt – stattdessen Muster

Ziel der Quellencodierung:

$$R_c \rightarrow 0 \Rightarrow L \approx H(X)$$

8.1.1. Huffman-Codierung

- Verfahren zur Entwicklung eines präfixfreien Codes mit minimaler mittlerer Codewortlänge L
- Rekursives Verfahren, d.h. der Binärbaum wird nicht von der Wurzel, sondern von den Blättern aus entwickelt

Kerngedanke

- Häufige Zeichen → kurze Codes
- Seltene Zeichen → lange Codes

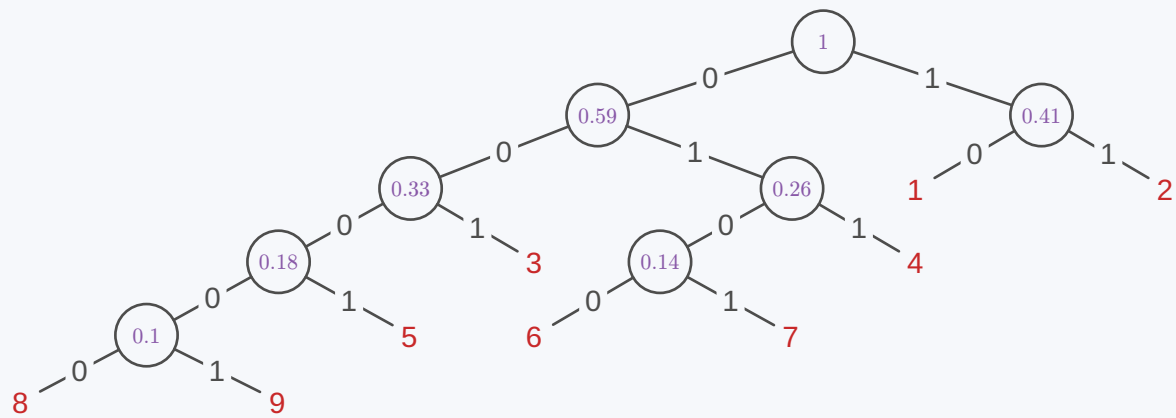
Verfahren

- 1) Sortiere Zeichen nach Wahrscheinlichkeit
- 2) Kombiniere die zwei kleinsten
- 3) Ersetze sie durch neuen Knoten
- 4) Wiederhole, bis ein Baum entsteht
- 5) Weise 0 / 1 entlang der Kanten zu

Der Huffman-Code ist nicht eindeutig – aber immer optimal (bzgl. mittlerer Länge).

Beispiel 35

x_i	1	2	3	4	5	6	7	8	9
$p(x_i)$	0.22	0.19	0.15	0.12	0.08	0.07	0.07	0.06	0.04



⇒

2 1 4 3 7 6 5 9 8
 11 10 011 001 0101 0100 0001 00001 00000

Entropie

TODO:

redundanz etc

$$H(X) \approx 2.89 \text{ Bit}$$

TODO:

bits

8.1.2. Run-Length-Encoding (RLE)

- Wiederholungen werden nicht mehrfach gespeichert, sondern gezählt.
- wird bei vielen Bildformaten benutzt zum Beispiel BMP, PCX und TIFF.
- gehört zu musterbasierten Verfahren
- Spezialfall von Lempel-Ziv

- sehr einfach
- sehr schnell
- keine Wahrscheinlichkeiten nötig

Kerngedanke

- Wiederholungen ersetzen durch: Symbol, Anzahl

TODO:

bits

Gut geeignet für

- lange Wiederholungen
- Bilder (z.B. einfarbige Flächen)
- einfache Muster

Schlecht geeignet für

- zufällige Daten

- häufige Wechsel

Beispiel 36

Quelltext	Codiert
$w = \text{Agggbbbehffgggg}$	$w_c = \text{A3g2beh3f4g}$
$ w = 15$	$ w_c = 11$

8.1.3. Lempel-Ziv-Codierung (LZ77)**TODO:**

bits

Kerngedanke

- Muster werden wiedererkannt und ersetzt durch Verweis auf vorherige Sequenz

Grundidee

Zwei Bereiche

- Search Buffer → Vergangenheit
- Look-Ahead Buffer → Zukunft

Grosser Buffer

Bessere Kompression, weil längere Muster erkennbar

Langsamer

Kleiner Buffer

Schneller, weil weniger Speicher

Schlechtere Kompression

Codierung

- Statt Zeichen: Distanz, Länge, Symbol
 - Distanz → wie weit zurück
 - Länge → wie lang das Muster
 - Symbol → nächstes Zeichen

Beispiel 37

Search Buffer							CP	Lookahead Buffer					Out
							a	b	r	a	c	ada...	(0,0,a)
						a	b	r	a	c	a	dab...	(0,0,b)
					a	b	r	a	c	a	d	abr...	(0,0,r)
				a	b	r	a	c	a	d	a	bra	(3,1,c)
		a	b	r	a	c	a	d	a	b	r	a	(2,1,d)
a	b	r	a	c	a	d	a	b	r	a			(7,4,eof)
c	a	d	a	b	r	a							
7	6	5	4	3	2	1							

Tokens

- Distanz = 3Bit
- Länge = 3Bit
- Symbol = 8Bit (ASCII)

Effizienzberechnung

- 14 Bit / Token: $6 \cdot 14 = 84$ Bit
- ASCII: $11 \cdot 8 = 88$ Bit

000	000	01100001	(0,0,a)
000	000	01100010	(0,0,b)
000	000	01110010	(0,0,r)
011	001	01100011	(3,1,c)
010	001	01100100	(2,1,d)
111	100	00000000	(7,4,eof)

8.1.4. Lempel-Ziv-Welch-Komprimierung (LZW)

Die Lempel-Ziv-Welch-Komprimierung ist ein Verfahren zur verlustfreien Datenkompression, das auf der Lempel-Ziv-Komprimierung aufbaut. Sie verwendet ein Wörterbuch, um wiederkehrende Sequenzen zu identifizieren und durch kürzere Codes zu ersetzen.

8.1.4.1. Funktionsweise

- 1) Beginne mit einem Wörterbuch, das bereits alle möglichen Einzelzeichen enthält.
- 2) Suche im Wörterbuch die längste Sequenz, die mit den nächsten Zeichen ($n + 1$) der Eingabe übereinstimmt.
- 3) Speichere den Index des gefundenen Wörterbucheintrags.
- 4) Erstelle einen neuen Eintrag im Wörterbuch mit der gefundenen Sequenz, gefolgt vom nächsten Zeichen der Eingabe.
- 5) Verschiebe das Eingabefenster um die Anzahl der codierten Zeichen.
- 6) Wiederhole den Prozess, bis alle Zeichen codiert sind.

Beispiel 38**Gegeben**

Zu kodierende Zeichenabfolge: 123123123123

Wörterbuch:

Index	Eintrag
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9

Vorgehen

<i>Buffer</i>	<i>Erkannte Zeichen (Index)</i>	<i>Index: Neuer Eintrag</i>	Neues Wörterbuch:	
123123123123	1 (1)	10: 12	<i>Index</i>	<i>Eintrag</i>
123123123123	2 (2)	11: 23	0-9	Wie bisher
123123123123	3 (3)	12: 31	10	12
123123123123	12 (10)	13: 123	11	23
123123123123	31 (12)	14: 312	12	31
123123123123	23 (11)	15: 231	13	123
123123123123	123 (13)	-	14	312
Daraus folgt die codierte Nachricht: 1 2 3 10 12 11 13			15	231

8.1.5. Kompressionsrate

$$R(\%) = \frac{L_{\text{komprimiert}}}{L_{\text{original}}} \cdot 100$$

L_{original} = Länge der ursprünglichen Sequenz

$L_{\text{komprimiert}}$ = Länge der komprimierten Sequenz

8.2. VERSCHLÜSSELUNG

Wenn eine Nachricht über einen offenen Kanal übertragen wird, entstehen sofort mehrere Probleme:

Vertraulichkeit: Nur der gewünschte Empfänger soll den Inhalt lesen können.

Integrität: Der Inhalt soll nicht unbemerkt verändert werden können.

Authentizität: Der Empfänger soll prüfen können, von wem die Nachricht stammt.

Ziel ist die gezielte Konstruktion einer leicht ausführbaren, aber schwer umkehrbaren Operation.

8.2.1. Substitutionsverfahren

– Caesar-Chiffre

8.2.1.1. Verschlüsselung

Ordnen wir Buchstaben Zahlen zu:

$$A = 0, B = 1, \dots, Z = 25$$

Dann gilt:

$$c = (m + k) \bmod 26$$

- m : Klartext
- k : Schlüssel
- c : Chiffretext

⇒ Interpretation: Addition in $\mathbb{Z}_{26}$

8.2.1.2. Entschlüsselung

$$m = c - k \bmod 26$$

⇒ Rückwärtsoperation ist trivial

Beispiel 39
 $m =$ bald sind sommerferien

 $k = 4$
 $c =$ feph wmrh wsqqivjvmir
8.2.1.3. Unsicher!

- Die statistischen Eigenschaften von Klar- und Chiffriertext sind nach wie vor unverändert
- Kennen wir die Sprache und ist unsere Probe gross genug, können wir den Schlüssel leicht ermitteln
- Schlüsselanzahl ist recht übersichtlich, hier braucht es keinen Computer, um alle auszuprobieren

8.2.2. Transpositionsverfahren

- Zeichen werden nicht verändert
- Nur ihre Position wird verändert

 $(x_1, x_2, \dots, x_n) \rightarrow (x_{\pi(1)}, x_{\pi(2)}, \dots, x_{\pi(n)}), \pi = \text{Permutation}$
Beispiel 40**Klartext**

DIE WORTE HOER ICH WOHL ALLEIN MIR
FEHLT DER GLAUBE

→ Erstellen einer Tabelle
zeilenweise

Chiffretext

DTILNHGIECAMLLEHHLITAWOWLRDUOE-
OEFEBRRHIERE

← Auslesen spaltenweise

D	I	E	W	O	R
T	E	H	O	E	R
I	C	H	W	O	H
L	A	L	L	E	I
N	M	I	R	F	E
H	L	T	D	E	R
G	L	A	U	B	E

8.2.2.1. Unsicher!

- Zeichenhäufigkeiten bleiben exakt erhalten
- Typische Muster bleiben sichtbar
- Statistische Analyse möglich

8.2.3. Polyalphabetisches Verfahren

- Vigenère-Chiffre

Nicht nur ein Alphabet sondern mehrere. Zuordnung hängt von der Position ab.

$$c_i = (m_i + k_i) \bmod 26$$

- m_i : i-tes Klartextzeichen (als Zahl)
- k_i : i-tes Schlüsselzeichen (als Zahl)
- c_i : i-tes Zeichen im Chiffretext

⇒ jedes Zeichen wird unterschiedlich verschlüsselt

Beispiel 41

TODO:

8.2.3.1. Unsicher!

- Der Schlüssel wird periodisch wiederholt, dadurch entstehen wiederkehrende Muster im Chiffretext
- Diese Muster erlauben es, die Schlüssellänge zu bestimmen

⇒ Der Text zerfällt in mehrere Teilfolgen → jede Teilfolge ist wieder eine Caesar-Chiffre

⇒ Mehr Komplexität – aber die gleiche Grundstruktur

⇒ Das Verfahren bleibt systematisch angreifbar

8.2.4. Probleme

Alle bisherigen Verfahren basieren auf:

- einfacher Algebra (Addition, Permutation)
- erkennbarer Struktur

⇒ Die Struktur ist zu einfach – und deshalb angreifbar.

8.2.5. Moderne symmetrische Verschlüsselung

	<i>DES (historisch)</i>	<i>AES (heute Standard)</i>
Chiffre	Blockchiffre (64 Bit Blöcke)	Blockchiffre (128 Bit Blöcke)
Schlüssel	56 Bit	128/192/256 Bit
Basiert auf	– Substitution (S-Boxen) – Permutation – Viele Runden (Feistel-Struktur)	– Substitution (S-Box) – Lineare Transformationen – Mehrere Runden
Sicherheit	Heute nicht mehr sicher (Schlüssel zu kurz)	Aktuell nicht gebrochen

⇒ Sicherheit entsteht durch komplexe, nichtlineare Transformationen

8.2.5.1. Warum moderne symmetrische Verfahren sicher sind

8.2.5.1.1. Eigenschaften moderner Blockchiffren

- Kombination vieler einfacher Operationen
- keine einfache algebraische Struktur sichtbar
- starke Durchmischung von Bits (Diffusion) führt zu Avalanche-Effekt: kleine Änderungen → grosse Auswirkungen

8.2.5.1.2. Angriffe

- keine einfachen statistischen Methoden mehr möglich
- nur noch:
 - brute force
 - sehr komplexe Spezialangriffe

⇒ Die Struktur ist so komplex, dass sie praktisch nicht mehr ausgenutzt werden kann

Angriffsmöglichkeiten

Brute Force:

- alle Schlüssel ausprobieren
- schauen, welcher sinnvollen Klartext ergibt

Brute Force ignoriert komplett:

- Struktur
- Mathematik

Differenzielle Kryptanalyse:

- Unterschiede im Input
- Unterschiede im Output

Analysieren, wie sich diese durch das System bewegen Ziel: nicht alle Schlüssel testen, sondern Information über den Schlüssel gewinnen

8.2.5.2. Probleme

- Sender und Empfänger brauchen denselben Schlüssel
 - dieser Schlüssel muss vorher ausgetauscht werden
 - Für n Teilnehmer werden $\binom{n}{2} = \frac{n(n-1)}{2}$ verschiedene Schlüssel benötigt
- quadratisches Wachstum (Schlüsselexplosion)

8.2.6. Asymmetrische Verschlüsselung

- Zwei unterschiedliche Schlüssel
- öffentlicher Schlüssel (darf bekannt sein)
- privater Schlüssel (bleibt geheim)

⇒ Wir lösen das Problem nicht durch mehr Komplexität, sondern durch ein neues mathematisches Konzept

8.2.6.1. RSA

Ziel:

- Verschlüsselung mit einem öffentlichen Schlüssel
- Entschlüsselung mit einem privaten Schlüssel

Eigenschaften:

- Jeder kann verschlüsseln
- Nur der Besitzer des privaten Schlüssels kann entschlüsseln
- Der private Schlüssel lässt sich aus dem öffentlichen nicht einfach berechnen

TODO:

Mult. Inv.

Für $a \in \mathbb{Z}_q$ ist $b \in \mathbb{Z}_q$ das **multiplikative inverse** von a , wenn $a \cdot b \equiv 1 \pmod{q}$

8.2.6.1.1. Berechnung

- 1) Wähle zwei Primzahlen p, q
- 2) Berechne $n = p \cdot q$
- 3) Berechne $\varphi(n) = (p-1)(q-1)$
- 4) Wähle a, b so, dass $a \cdot b \equiv 1 \pmod{\varphi(n)}$
- 5) Vergesse $p, q, \varphi(p \cdot q)$. Brauchen wir nicht und riskieren nur, dass uns jemand hackt

Public key ist nun n, b , Private key ist n, a

- Verschlüsseln: $z = c^a \pmod{n}$
- Entschlüsseln: $c = z^b \pmod{n} = c^{a \cdot b} \pmod{n}$

8.2.6.1.2. Satz von Euler

Sei $n \in \mathbb{N} \setminus \{0\}$ und $z \in \mathbb{Z}$ mit $\text{ggT}(z, n) = 1$. Dann ist $z^{\varphi(n)} \equiv 1 \pmod{n}$.

8.2.6.1.2.1. Euler'sche φ -Funktion

Sei $n \in \mathbb{N} \setminus \{0\}$ und $\mathbb{Z}_n^* = \{x \in \mathbb{Z}_n \mid x \text{ hat ein multiplikatives Inverses in } \mathbb{Z}_n\}$.
Dann heisst $\varphi(n)$:

$$\begin{aligned}\varphi(n) &= \text{Anz. Elemente in } \mathbb{Z}_n \text{ mit mult. Inversen} \\ &= \text{Anz. Zahlen } 1 \leq q \leq n \text{ mit } \text{ggT}(q, n) = 1 \\ &= |\mathbb{Z}_n^*|\end{aligned}$$

Rechenregeln

- 1) Sei $n \in \mathbb{N}$ eine Primzahl, dann $\varphi(n) = n - 1$
- 2) Sei $n \in \mathbb{N}$ eine Primzahl und $p \in \mathbb{N} \setminus \{0\}$, dann $\varphi(n^p) = n^{p-1} \cdot (n - 1)$
- 3) Seien $m, n \in \mathbb{N} \setminus \{0\}$ und $\text{ggT}(m, n) = 1$, dann $\varphi(n \cdot m) = \varphi(n) \cdot \varphi(m)$

8.2.6.1.3. Erweiterter Euklidischer Algorithmus

Seien $a, b \in \mathbb{N}, a \neq b, a \neq 0, b \neq 0$

Initialisierung: Setze $x := a, y := b, q := x \div y, r := x - q \cdot y, (u, s, v, t) = (1, 0, 0, 1)$ (d.h. bestimme q und r so, dass $x = q \cdot y + r$ ist)

Wiederhole bis $r = 0$ ist

Ergebnis: $y = \text{ggT}(a, b) = s \cdot a + t \cdot b$

Wenn $\text{ggT}(a, b) = 1$ ist, dann folgt: $t \cdot v \equiv 1 \pmod{a}$

Beispiel 42 : $\text{ggT}(99, 79)$

i	$x = y_{-1}$	$y = r_{-1}$	$q = x \div y$	$r = x - q \cdot y$	$u = s_{-1}$	$s = u_{-1} - q_{-1} \cdot s_{-1}$	$v = t_{-1}$	$t = v_{-1} - q_{-1} \cdot t_{-1}$
$i = 0$	99	79	1	20	1	0	0	1
$i = 1$	79	20	3	19	0	1	1	-1
$i = 2$	20	19	1	1	1	-3	-1	4
$i = 3$	19	1	19	0	-3	4	4	-5

Daraus folgend:

- $\text{ggT}(99, 79) = 4 \cdot 99 + (-5) \cdot 79 \Leftrightarrow 396 - 395 = 1$
- $99 + (-5) = 94$ ist mult. Inv. von 79 in $\mathbb{Z}_{99}$
- $79 + 4 = 83 \equiv 4$ ist mult. Inv. von 99 in $\mathbb{Z}_{79}$

TODO:

CySec: Chiffres

8.2.6.2. Grösse

1024-Bit RSA: zwei Primzahlen mit je 512 Bit

- Anzahl möglicher Primzahlen: $\approx 10^{151}$

2048-Bit RSA: zwei Primzahlen mit je 1024 Bit

- Anzahl möglicher Primzahlen: $\approx 10^{305}$

4096-Bit RSA: zwei Primzahlen mit je 2048 Bit

- Anzahl möglicher Primzahlen: $\approx 10^{613}$

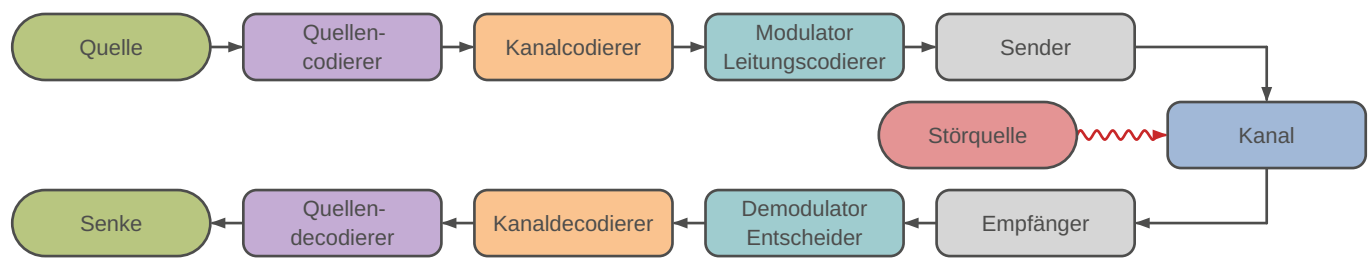
Die Gesamtzahl der möglichen Kombinationen von zwei Primzahlen aus 10^{151} Primzahlen (1024-Bit RSA-Schlüssel) ist gegeben durch die Kombination ohne Wiederholung:

$$t = \frac{\text{Kombinationen}}{\text{Faktorisierungen pro Sekunde}} = \frac{\frac{1}{2}(10^{151})^2 s}{100 \cdot 10^9} = \frac{10^{302} s}{2 \cdot 10^{11}} \approx 10^{290} s$$

9. KANALMODELL

Mathematisches Modell, das beschreibt, wie Informationen durch einen Kommunikationskanal übertragen werden.

Kanal: Medium der Übertragung

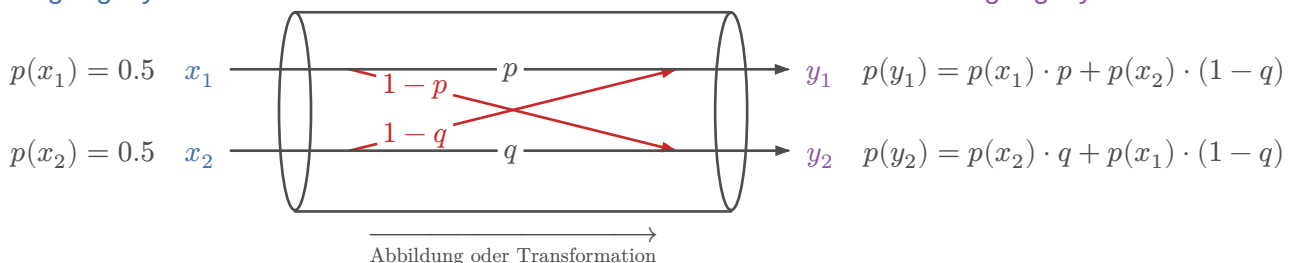


9.1. KANALMATRIX

Auch **Übergangsmatrix** genannt. Gibt an, wie die Eingangssignale (gesendete Symbole) durch den Kanal in Ausgangssignale (empfangene Symbole) überführt werden. Diese Matrix beinhaltet die bedingten Wahrscheinlichkeiten $P(Y|X)$, wobei jedes Element der Matrix die Wahrscheinlichkeit darstellt, dass ein bestimmtes Ausgangssymbol y empfangen wird, gegeben ein gesendetes Eingangssymbol x .

Eingangssymbol

Ausgangssymbol



Gegeben x

$$P(Y|X) = \begin{pmatrix} p & 1-p \\ 1-q & q \end{pmatrix}, \text{ Rot} \rightarrow \text{Fehlerwahrscheinlichkeit}$$

Resultierendes y

$$\begin{aligned}
 P(Y|X) &= \underbrace{\begin{pmatrix} p(y_1|x_1) & p(y_2|x_1) \\ p(y_1|x_2) & p(y_2|x_2) \end{pmatrix}}_{\text{Kanalmatrix}} \\
 p(y_1) &= p(x_1) \cdot p(y_1|x_1) + p(x_2) \cdot p(y_1|x_2) \\
 p(y_2) &= p(x_1) \cdot p(y_2|x_1) + p(x_2) \cdot p(y_2|x_2)
 \end{aligned}$$

Um die Kanalmatrix eines Kanals zu ermitteln, sendet man wiederholt ein bekanntes Zeichen, zeichnet die Empfänge auf und berechnet aus diesen Daten die Häufigkeiten, um die Matrix zu erstellen.

Kanalkapazität (für BSC): $C = 1 - H(Y|X)$ = Maximale Informationsrate, bei der Fehler gegen 0 möglich sind

- $R < C$: Fehlerfreie Übertragung möglich
- $R > C$: Fehler unvermeidbar

→ Redundanz hinzufügen, um korrekte Übertragung sicherzustellen

9.1.1. Generalisierung

$$P(Y|X) = \underbrace{\begin{pmatrix} p(y_1|x_1) & p(y_2|x_1) & \dots & p(y_n|x_1) \\ p(y_1|x_2) & p(y_2|x_2) & \dots & p(y_n|x_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(y_1|x_m) & p(y_2|x_m) & \dots & p(y_n|x_m) \end{pmatrix}}_{\text{Spalten = empfangene Symbole}} \left. \vphantom{\begin{pmatrix} p(y_1|x_1) & p(y_2|x_1) & \dots & p(y_n|x_1) \\ p(y_1|x_2) & p(y_2|x_2) & \dots & p(y_n|x_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(y_1|x_m) & p(y_2|x_m) & \dots & p(y_n|x_m) \end{pmatrix}} \right\} \text{Zeilen = gesendete Symbole}$$

$$P(Y) = P(X)^T \cdot P(Y|X)$$

$$p(y_k) = \sum_{i=1}^m p(x_i) \cdot p(y_k|x_i)$$

9.1.2. Nicht gestörter Kanal

= Einheitsmatrix

$$P(Y|X) = \mathbb{1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

9.1.3. Vollständig gestörter Kanal

= Matrix, bei der alle Elemente gleich sind

$$P(Y|X) = \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{pmatrix}$$

9.1.4. Verlustfreie (rauschbehaftete) Matrix

Summe jeder Zeile muss 1 ergeben.

$$P(Y|X) = \begin{pmatrix} 0.7 & 0.3 \\ 0.9 & 0.1 \end{pmatrix}$$

TODO:

example (slides 12)

9.2. MAXIMUM-LIKELIHOOD-VERFAHREN (ML)

Die Entscheidungsfindung (Detektion) nach dem Maximum-Likelihood-Verfahren wählt (pro Spalte) für jedes empfangene Symbol y_j das wahrscheinlichste gesendete Symbol x_i basierend auf der Kanalmatrix. Die Zuordnung erfolgt durch:

$$\hat{x}_{\text{ML}} = \arg \max_{x_i} p(y_i|x_i)$$

TODO:

slides 13

Beispiel 43

$$P(Y|X) = \begin{pmatrix} 0.2 & \mathbf{0.5} & 0.3 \\ \mathbf{0.7} & 0.2 & 0.1 \\ 0.4 & 0 & \mathbf{0.6} \end{pmatrix} \Rightarrow \begin{cases} y_1 \rightarrow x_2 \\ y_2 \rightarrow x_1 \\ y_3 \rightarrow x_3 \end{cases}$$

9.3. MAXIMUM-A-POSTERIORI-VERFAHREN (MAP)

$$\hat{x}_{\text{MAP}} = \arg \max_{x_i} p(x_i) \cdot p(y_i|x_i)$$

TODO:

slides 14

Beispiel 44

TODO:

9.4. ENTROPIE

Beispiele bauen auf folgenden Daten auf, falls nicht explizit erwähnt:

$$P(Y|X) = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$$

$$p(x_1) = 0.3, p(x_2) = 0.7$$

$$\text{Übertragungsrate} = 1 \frac{\text{kbit}}{\text{s}}$$

TODO:

slides 16, 17

9.4.1. Eingangswahrscheinlichkeit

$$p(x_i) = \frac{p(x_i|y_j) \cdot p(y_j)}{p(y_j|x_i)}$$

Beispiel 45

TODO:

9.4.2. Ausgangswahrscheinlichkeit

$$p(y_j) = \sum_{i=1}^m p(x_i) \cdot p(y_j|x_i)$$

Beispiel 46

$$p(y_1) = 0.3 \cdot 0.9 + 0.7 \cdot 0.2 = 0.41$$

$$p(y_2) = 0.3 \cdot 0.1 + 0.7 \cdot 0.8 = 0.59$$

9.4.3. Gemeinsame Entropie / Verbundentropie

$$\begin{aligned} H(X, Y) &= - \sum_i^m \sum_j^n p(x_i, y_j) \cdot \log_2 p(x_i, y_j) \\ &= - \sum_i^m \sum_j^n p(x_i) \cdot p(x_i | y_j) \cdot \log_2 (p(x_i) \cdot p(x_i | y_j)) \end{aligned}$$

Beispiel 47

$$\begin{aligned} H(X, Y) &= -(0.3 \cdot 0.9 \cdot \log_2(0.3 \cdot 0.9) + 0.3 \cdot 0.1 \cdot \log_2(0.3 \cdot 0.1) \\ &\quad + 0.7 \cdot 0.2 \cdot \log_2(0.7 \cdot 0.2) + 0.7 \cdot 0.8 \cdot \log_2(0.7 \cdot 0.8)) \\ &\approx 1.527339244 \end{aligned}$$

TODO:

check

9.4.4. Entropie am Kanaleingang

$$H(X) = - \sum_{i=1}^m p(x_i) \cdot \log_2 p(x_i)$$

Beispiel 48

$$\begin{aligned} H(X) &= -(0.3 \cdot \log_2(0.3) + 0.7 \cdot \log_2(0.7)) \\ &\approx 0.8812908992 \text{ bit/Zeichen} \end{aligned}$$

9.4.5. Entropie am Kanalausgang

Durchschnittliche Unsicherheit der empfangenen Symbole.

$$H(Y) = - \sum_{j=1}^n p(y_j) \cdot \log_2 p(y_j)$$

Beispiel 49

$$\begin{aligned} H(Y) &= -(0.41 \cdot \log_2(0.41) + 0.59 \cdot \log_2(0.59)) \\ &\approx 0.9765004688 \text{ bit/Zeichen} \end{aligned}$$

9.4.6. Äquivokation vs. Irrelevanz

Äquivokation

Empfänger → Sender

Irrelevanz

Sender → Empfänger

Äquivokation	Irrelevanz
$H(X Y)$: Wie unsicher ist das gesendete Signal X , obwohl das empfangene Signal Y bekannt ist?	$H(Y X)$: Wie unsicher ist das empfangene Signal Y , obwohl das gesendete Signal X bekannt ist?
beschreibt den Verlust an Information	beschreibt das Rauschen im Kanal
ein Ausgang kann von mehreren Ursachen stammen	ein Eingang führt zu mehreren möglichen Ausgängen

9.4.7. Äquivokation

$H(X|Y)$ misst die verbleibende Unsicherheit über das tatsächliche X , nachdem Y bekannt ist. Auch **Rückschlusssentropie** genannt. Ist der Kanal fehlerfrei, so ist $H(X|Y) = 0$.

TODO:
check

$$\begin{aligned}
 H(X|Y) &= - \sum_i^m \sum_j^n p(x_i, y_j) \cdot \log_2 p(x_i|y_j) \\
 &= - \sum_i^m \sum_j^n p(x_i) \cdot p(x_i|y_j) \cdot \log_2 p(x_i|y_j) \\
 &= H(X, Y) - H(Y)
 \end{aligned}$$

Beispiel 50

9.4.8. Irrelevanz

$H(Y|X)$ misst die verbleibende Unsicherheit über das Empfangssignal Y , obwohl das gesendete Signal X bekannt ist. Auch **Streuentropie** genannt. Ist der Kanal fehlerfrei, so ist $H(Y|X) = 0$.

$$\begin{aligned}
 H(Y|X) &= - \sum_i^m \sum_j^n p(x_i, y_j) \cdot \log_2 p(y_j|x_i) \\
 &= - \sum_i^m \sum_j^n p(x_i) \cdot p(y_j|x_i) \cdot \log_2 p(y_j|x_i) \\
 &= H(X, Y) - H(X)
 \end{aligned}$$

Beispiel 51

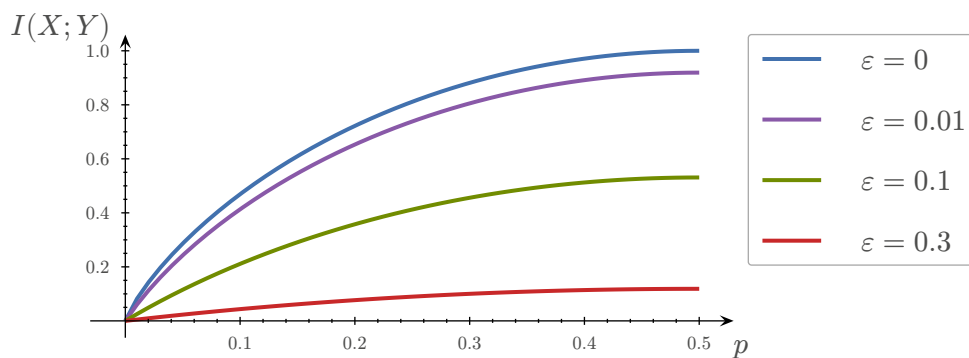
$$\begin{aligned}
 H(Y|X) &= -(0.3 \cdot 0.9 \cdot \log_2(0.9) + 0.3 \cdot 0.1 \cdot \log_2(0.1) \\
 &\quad + 0.7 \cdot 0.2 \cdot \log_2(0.2) + 0.7 \cdot 0.8 \cdot \log_2(0.8)) \\
 &\approx 0.6460483445
 \end{aligned}$$

9.4.9. Transinformation

Auch **gegenseitige Information** genannt. Gibt den maximalen und somit fehlerfreien Informationsfluss über einen gestörten Kanal an.

- Bei vollständig gestörtem Kanal ist $T = 0$, d.h. $H(Y) = H(Y|X) = 1$ und zwar unabhängig von der Entropie am Kanaleingang.
- Verändert sich die Entropie der Quelle, so verändert sich auch die Transinformation.
- Nimmt die Fehlerwahrscheinlichkeit zu, so verringert sich die Transinformation.
- Ein nicht gestörter Kanal (Einheitsmatrix) überträgt den mittleren Informationsfluss ohne weiteren Verlust, d.h. die Transinformation wird nur durch die Quelle bestimmt.

$$\begin{aligned}
 I(X; Y) &= \text{Ursprüngliche Information} - \text{Verlust} = H(X) - H(X|Y) \\
 &= \text{Empfang} - \text{Rauschen} = H(Y) - H(Y|X) \\
 &= \sum_i^m \sum_j^n p(x_i, y_j) \cdot \log_2 \left(\frac{p(y_j|x_i)}{p(y_j)} \right) \\
 &= \sum_i^m \sum_j^n p(x_i, y_j) \cdot \log_2 \left(\frac{p(x_i|y_j)}{p(x_i)} \right)
 \end{aligned}$$



Beispiel 52

$$I(X; Y) = H(Y) - H(Y|X) = 0.9765004688 - 0.6460483445 = 0.3304521243$$

9.4.10. Maximale Symbolrate

$$\begin{aligned}
 R_{\max} \left[\frac{\text{Zeichen}}{s} \right] &= \text{Übertragungsrate} \cdot I(X; Y) \\
 &= \text{Bandbreite des Kanals} \cdot H(X)
 \end{aligned}$$

Beispiel 53

$$R_{\max} \left[\frac{\text{Zeichen}}{s} \right] = 0.3304521243 \cdot 1000 = 330.4521243$$

TODO:

summary (slides 22)

TODO:

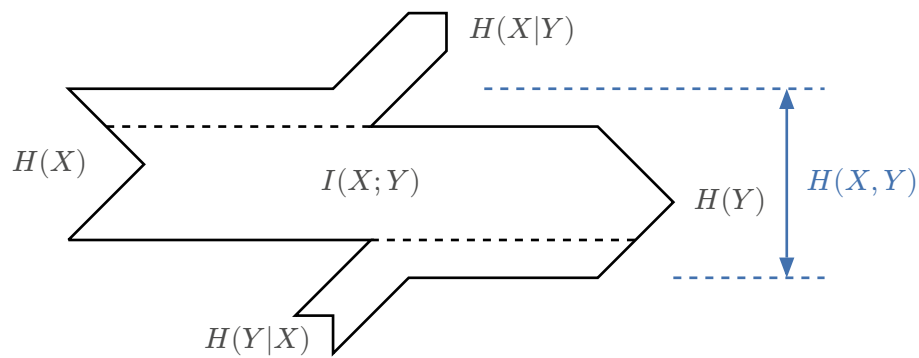
diagram (slides 23)

TODO:

example (slides 24-27)

9.4.11. Zusammenhänge

9.4.11.1. Visuell



9.4.11.2. Rechnerisch

$$\begin{aligned}
 H(X) &\geq H(X|Y) \\
 H(Y) &\geq H(Y|X) \\
 H(X, Y) &= H(X) + H(Y|X) \\
 &= H(Y) + H(X|Y) \\
 &= H(X) + H(Y) - I(X; Y) \\
 I(X; Y) &\geq 0
 \end{aligned}$$

9.5. FEHLERERKENNUNG UND FEHLERKORREKTUR

9.5.1. Blockcodes

Ein Blockcode C teilt das eingehende Nachrichtensignal in gleich lange Blöcke der Länge m auf und erzeugt daraus Blöcke der Länge n , wobei zusätzliche Redundanz beigefügt wird und damit $n > m$ ist.

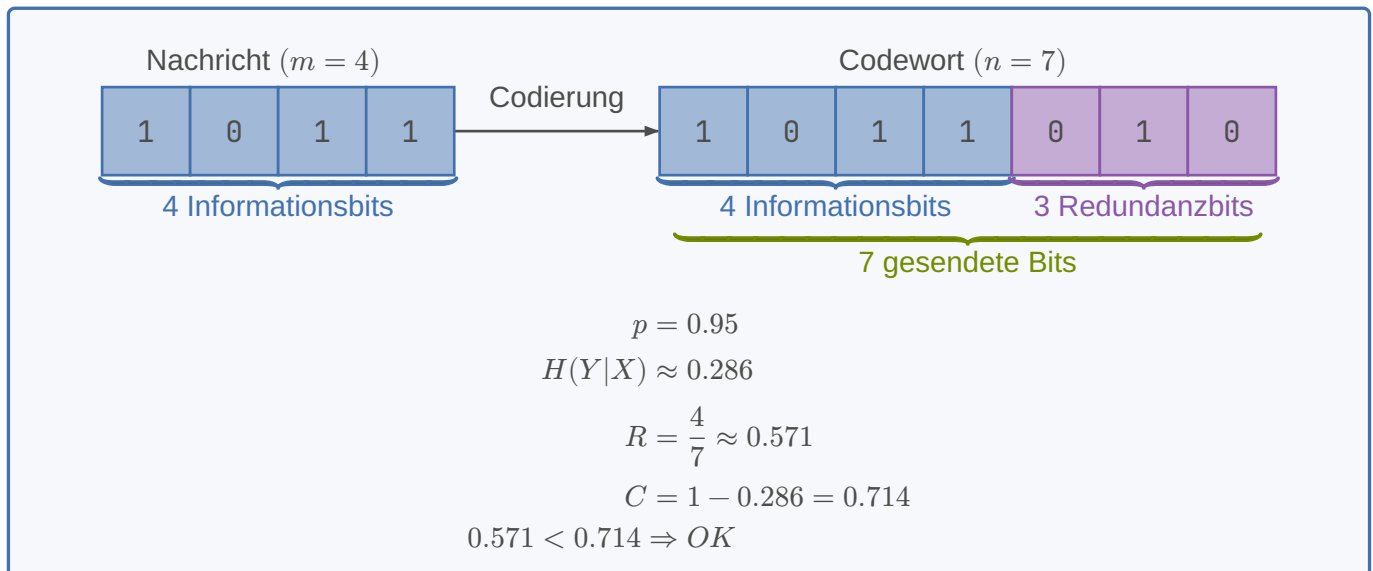
Beispiel 54

	gültig:			ungültig:		
	$m = 2$		$k = 1$	$m = 2$		$k = 1$
$m = 2$	0	0	0	0	0	1
$k = 1$	0	1	1	0	1	0
$n = 3$	1	0	1	1	0	0
	1	1	0	1	1	1

9.5.2. Coderate

Die Coderate R_c eines binären (n, m) -Blockcodes C ist wie folgt definiert: $R_c = \frac{m}{n}$ und beschreibt, wie gross der Anteil der Nutzdaten in den Codeworten von C sind.

Beispiel 55



9.5.3. Coderaum

TODO:

(slides 9-11)

9.5.4. Hamming-Gewicht und Hamming-Distanz

Das Hamming-Gewicht $w(c)$ eines Codewortes c entspricht der Anzahl Elemente, welche im Codevektor ungleich Null sind. Bei einem binären Code C entspricht das Hamming-Gewicht somit der Anzahl Einer im Codewort c .

Die Hamming-Distanz misst die Anzahl unterschiedlicher Positionen zwischen zwei gleich langen Codewörtern. Sie wird genutzt, um die Fähigkeit eines Codes zur Fehlererkennung und -korrektur zu bewerten.

TODO:

Hammingdistanz: $h = d_{\min} = \min_{i,j} (d(x_i, x_j))$

Beispiel 56

00000000 Kleinste Distanz $h = 2$
 11000000
 10001100
 01010000
 01010101
 10000110
 11111111

9.5.5. Fehlererkennung

Anzahl der sicher erkennbaren Fehler: $e^* = d_{\min} - 1$

C ist r -fehlererkennend $\Leftrightarrow d_{\min} > r$

TODO:

slides 14

9.5.6. Fehlerkorrektur

Anzahl der sicher korrigierbaren Fehler: $e = \left\lfloor \frac{d_{\min}-1}{2} \right\rfloor$

$d_{\min} \geq 2e + 1$ notwendig für eindeutige Fehlerkorrektur

C ist r -fehlerkorrigierend $\Leftrightarrow d_{\min} > 2r$

TODO:

slides 15

9.5.7. 1D-Parity

Idee: Ein zusätzliches Bit macht die Gesamtzahl der 1en gerade (even parity).

Eigenschaften:

- Länge: $n = k + 1$
- Mindestabstand: $d_{\min} = 2$

Kann:

- ✓ 1-Bit-Fehler erkennen
- ✗ keinen Fehler korrigieren
- ✗ 2-Bit-Fehler nicht zuverlässig erkennen

Beispiel: 101 $\rightarrow$ 1010

9.5.8. 2D-Parity

Idee: Parität in zwei Dimensionen (Zeile + Spalte).

Daten $r \times c \rightarrow$ gesendet $(r + 1) \times (c + 1)$

Eigenschaften:

- Produkt zweier Paritätscodes
- Mindestabstand: $d_{\min} = 4$

Kann:

- ✓ 1-Bit-Fehler erkennen
- ✓ bis 3 Bitfehler erkennen
- ✗ bestimmte 4-Bit-Fehler (Rechteck) bleiben unentdeckt

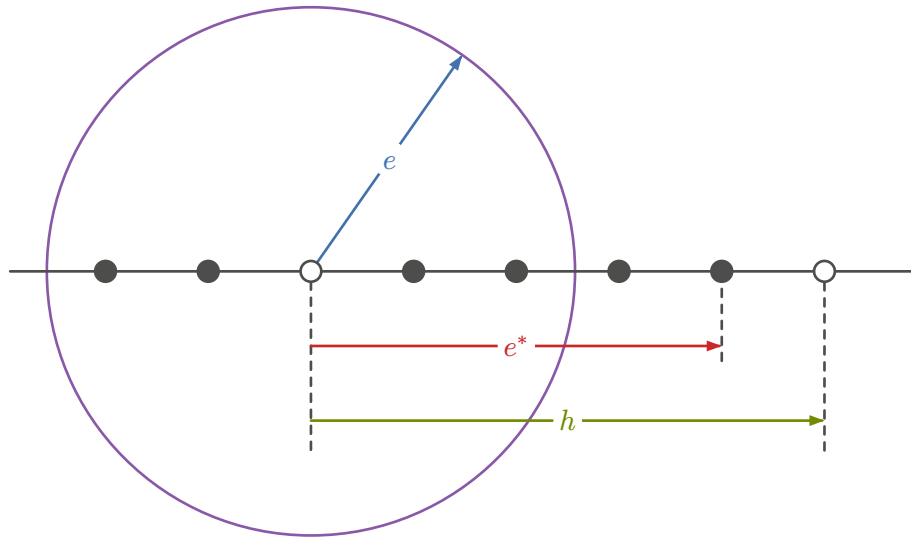
Beispiel 57

x_{11}	x_{12}	x_{13}	...	x_{1n}	$p_{1(n+1)}$	Sicher erkennbare Fehler: 3 Sicher korrigierbare Fehler: 1 Hammingdistanz: 4 (Konstant)
x_{21}	x_{22}	x_{23}	...	x_{2n}	$p_{2(n+1)}$	
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	
x_{k1}	x_{k2}	x_{k3}	...	x_{kn}	$p_{k(n+1)}$	
$p_{(k+1)1}$	$p_{(k+1)2}$	$p_{(k+1)3}$	...	$p_{(k+1)n}$	$p_{(k+1)(n+1)}$	

9.5.9. Korrigierkugeln

Illustriert die Fehlertoleranz eines Codes, und zeigt visuell, wie viele Fehler innerhalb eines Codewortes korrigiert werden können.

- Zentrum der Korrigierkugel: Liegt bei jedem gültigen Codewort.
- Radius der Korrigierkugel e : Maximale Anzahl an Fehlern, die sicher korrigiert werden können.
- Hammingdistanz h : Minimale Anzahl von Stellen, in denen sich zwei beliebige gültige Codewörter unterscheiden. Sie dient als Mass für die Fehlererkennungs- und -korrekturfähigkeit eines Codes.
- Sicher erkennbare Fehler e^* : Maximale Anzahl von Fehlern, die sicher erkannt werden können.
- Ungültige Codewörter: Liegen innerhalb der Korrigierkugel und stellen mögliche fehlerhafte Zustände des Codeworts dar, die zu einem gültigen Codewort korrigiert werden können.

**TODO:**

slides 19,20

9.5.10. Mögliche / gültige Codewörter

- 2^m = Anzahl gültigen Codewörter (Anzahl der Spalten der Matrix ohne Einheitsmatrix)
- 2^{m+k} = Anzahl möglicher Codewörter (Grösse der Einheitsmatrix)
- m = Nachrichtenstellen
- k = Kontrollstellen

9.5.11. Hamming-Schranke

- n = die Dimension des Codes (Anzahl aller CW = 2^n),
- m = die Dimension der Nachrichten (Anzahl aller gültigen CW = 2^m)
- k = die Dimension der Kontrollstellen mit $n = m + k$

Schranke:

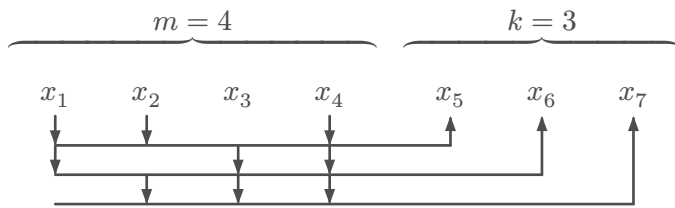
$$\underbrace{2^m}_{\text{Anzahl gültige CW}} \cdot \underbrace{\sum_{w=0}^e \binom{n}{w}}_{\text{Anzahl CW pro Korrigierkugel}} \leq \underbrace{2^n}_{\text{Anzahl aller CW}}$$

- Lücken → ineffizient
- Exakt gefüllt → optimal

Gilt

$$2^m \cdot \sum_{w=0}^e \binom{n}{w} = 2^n$$

so ist der Code dichtgepackt

TODO:**9.5.12. Kontrollmatrix und Codebedingung**

Kontrollmatrix:

$$\left(\begin{array}{cccc|ccc} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{array} \right)$$

$$\vec{P}_1 \vec{P}_2 \vec{P}_3 \vec{P}_4 \vec{P}_5 \vec{P}_6 \vec{P}_7$$

$$x_5 = (x_1 + x_2 + x_4) \bmod 2$$

$$x_6 = (x_1 + x_3 + x_4) \bmod 2$$

$$x_7 = (x_2 + x_3 + x_4) \bmod 2$$

Codebedingung:

$$\sum_i x_i \cdot \vec{P}_i \equiv \vec{0} \bmod 2$$

TODO:

slides 29-31

Beispiel 58 : Berechnung der Kontrollstellen und Fehlersyndrome

$$\text{Hamming blockcode} = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\text{Codewort} = 10110000010$$

$$\begin{aligned} x_{12} &\equiv (x_1 + x_2 + x_3 + x_4 + x_6 + x_7 + x_8) \bmod 2 \\ &\equiv (1 + 0 + 1 + 1 + 0 + 0 + 0) \equiv 1 \end{aligned}$$

$$\begin{aligned} x_{13} &\equiv (x_1 + x_2 + x_3 + x_5 + x_6 + x_9 + x_{10}) \bmod 2 \\ &\equiv (1 + 0 + 1 + 0 + 0 + 0 + 1) \equiv 1 \end{aligned}$$

$$\begin{aligned} x_{14} &\equiv (x_1 + x_2 + x_4 + x_5 + x_7 + x_9 + x_{11}) \bmod 2 \\ &\equiv (1 + 0 + 1 + 0 + 0 + 0 + 0) \equiv 0 \end{aligned}$$

$$\begin{aligned} x_{15} &\equiv (x_1 + x_3 + x_4 + x_5 + x_8 + x_{10} + x_{11}) \bmod 2 \\ &\equiv (1 + 1 + 1 + 0 + 0 + 1 + 0) \equiv 0 \end{aligned}$$

$$\text{Kontrollstellen} = 1100$$

$$\text{Codewort} = 10110000010\mathbf{1100}$$

$$\text{Fehlersyndrome f\"ur } x_2 = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 0 \end{pmatrix}, x_{11} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}, x_2 \wedge x_{11} = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

9.5.13. Konstruktion g\"ultiger Codew\"orter**TODO:**

9.5.14. Fehlersyndrom

Das Fehlersyndrom wird verwendet, um die Position eines Fehlers in einem Codewort zu identifizieren. Es ist das Ergebnis der Multiplikation des empfangenen Vektors mit der transponierten Prüfmatrix. Ein Fehlersyndrom, das ungleich dem Nullvektor ist, deutet auf das Vorhandensein eines oder mehrerer Fehler im Codewort hin.

$$\vec{Z} = \sum_i x_i \cdot \vec{P}_i \bmod 2$$

TODO:

slides 29-31

10. QUBIT

- Superposition
 - Ein Qubit kann gleichzeitig 0 und 1 repräsentieren. Mehrere Qubits können dadurch alle 2^n möglichen Zustände gleichzeitig darstellen.
 - $|\Psi\rangle = \alpha |0\rangle + \beta |1\rangle$
 - $P(0) = |\alpha|^2, P(1) = |\beta|^2$
- Beispiel
 - $|\Psi\rangle = \sqrt{0.75} |0\rangle + \sqrt{0.25} |1\rangle$
 - $P(0) = 0.75, P(1) = 0.25$
- Interferenz
 - α, β sind Amplituden. Zwei Qubits können sich gegenseitig beeinflussen, indem die Amplituden mittels Interferenz verstärkt oder ausgelöscht werden
 - Quantenalgorithmen verändern gezielt die Amplituden:
 - richtige Lösungen werden verstärkt
 - falsche Lösungen werden abgeschwächt
- Verschränkung
 - Verschränkung bedeutet, dass zwei Qubits einen gemeinsamen Zustand besitzen, der sich nicht in zwei unabhängige Einzelzustände zerlegen lässt.
 - Misst man eines der Qubits, ist der Zustand des anderen sofort festgelegt

11. CPU

Gesamtübersicht des CPU Zyklus

- 1) Fetch: Instruktion aus RAM lesen
- 2) Decode: Opcode interpretieren
- 3) Operand-Fetch: Speicherwerte laden
- 4) Execute: ALU rechnet
- 5) Writeback: Ergebnis ins Register
- 6) IP erhöhen

BIBLIOGRAFIE

- [1] D. Bühler, «Zusammenfassung DigCod». Zugegriffen: 3. Mai 2026. [Online]. Verfügbar unter: <https://studentenportal.ch/dokumente/digcod/2828/>